

claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a9cc4fa4940bf241f.jsonl`  
- SHA-256: `79049a94e552b95bd01a3a07607fd285454c0ae6c187224bec5500ca9c98ee7a`  
- Source modified: `2026-07-08T06:20:03+00:00`  
- Imported at: `2026-07-08T15:59:28+00:00`  
- project: `wf_5f13fc79-bad`  
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`
```

Transcript

1. user (2026-07-08T06:16:23.441Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).

The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):

- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing, never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (medium/silent-data-loss) backend/orchestration.py:1308

summary: A cloud-dispatched compile with a non-URI storage.output_root silently discards the reel yet reports success, because reel_object_uri() returns "" on the worker and nothing guards for it.

failure scenario: Config deployment.compute_target=cloud_run (so _cloud_available()==True regardless of output_root, orchestration.py:209) + phase_placement.compile=cloud_run_l4 + storage.output_root=/data/reels (a non-URI local path, which passes config validation). A compile job runs: _execute_compile -> _dispatch_compile_remote. At line 1307-1308 out_root="/data/reels" is truthy so compute is built and the job dispatches (only truthiness is checked, never URI-ness). On the L4 worker cmd_compile forwards storage.output_root=/data/reels; _stitch_reel computes reel_uri = reel_object_uri(cfg,...) which returns "" because there is no "://" (orchestration.py:1154), so the mirror-upload at line 1256-1264 is SKIPPED and the reel stays only in the worker's scratch dir, discarded when the Job exits. The worker still writes a marker status=success/rc=0, so run_compile returns rc=0 and _dispatch_compile_remote returns {status:success, download_url:/api/highlights/<vid>/<name>}. The SPA shows success; the download then 404s (the reel exists on no reachable disk). Unlike the in-process path, "" is NOT a valid local-serve on a remote worker. The dispatch test masks this (tests/test_compile_dispatch.py:91 uses a local output_root but mocks the compute, so the real worker mirror path is never exercised).

2. assistant (2026-07-08T06:16:32.828Z)

I'll investigate this claim by reading the actual code. Let me start with the key files.

3. user (2026-07-08T06:16:34.301Z)

```
1      """Phase ? machine placement resolution (#521).
2
3      A *phase placement* answers one question per pipeline phase ? **where does the heavy
compute for
4      this phase run? ** ? with a config knob (`deployment.phase_placement`) instead of code.
The three
5      placeable phases and their values:
6
7          validation ? control_plane | cloud_run_l4
8          segment    ? control_plane | cloud_run_l4    (a.k.a. "detect")
9          compile     ? control_plane | cloud_run_l4    (a.k.a. "stitch")
10
11      ``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
``cloud_run_l4``
12      dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already
uses). This module
13      is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the
worker never
14      re-derive the mapping.
15
16      **Default = today's behaviour (pure / additive).** When a phase's knob is unset
(``None``) the
17      resolver reproduces the pre-#521 placement:
18
19          validation ? control_plane                                (the authoritative
gate, #512/#513)
20          segment    ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
21          compile     ? control_plane                                (the in-process
ffmpeg stitch)
22
23      so a config with no ``phase_placement`` block behaves byte-identically to before this
module existed.
24      """
25
26      from __future__ import annotations
27
28      from typing import Any
29
30      # Phase identities (kept as constants so callers don't stringly-type the phase names).
```

```

31     PHASE_VALIDATION = "validation"
32     PHASE_SEGMENT = "segment"
33     PHASE_COMPILE = "compile"
34     PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
35
36     # Placement targets.
37     PLACE_CONTROL_PLANE = "control_plane"
38     PLACE_CLOUD_L4 = "cloud_run_l4"
39     PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
40
41
42     def resolve_phase_placement(cfg: Any, phase: str) -> str:
43         """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
44
45         Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the
46         legacy default
47         (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
48         ``deployment`` / ``phase_placement`` degrades to the legacy default.
49
50         deployment = getattr(cfg, "deployment", None)
51         placement = getattr(deployment, "phase_placement", None)
52         explicit = getattr(placement, phase, None) if placement is not None else None
53         if explicit:
54             return str(explicit)
55         # Legacy defaults ? reproduce today's placement exactly.
56         if phase == PHASE_SEGMENT:
57             compute_target = (getattr(deployment, "compute_target", "") or "").strip()
58             return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
59         # validation + compile default to the control-plane (their pre-#521 home).
60         return PLACE_CONTROL_PLANE

```

4. user (2026-07-08T06:16:34.793Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1193 of 1524 total (27133 tokens, cap 25000). Call Read with offset=1194 limit=1193 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
2     #390).
3
4     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
5     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
6     ``_ingest_remote_segments``),
7     and the shared compile resolution + worker handler (``_resolve_compile`` /
8     ``_execute_compile``).
9     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
10    unchanged.
11
12    SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
13    remain
14    monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
15    ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
16    through
17    the ``backend.main`` module object at call time (``import backend.main as _main``),
18    never as a
19    bare local. ``backend.main`` re-exports every name below, so
20    ``monkeypatch.setattr(backend.main,
21    ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
22    unchanged.
23
24    This module must NOT import ``backend.main`` at module level (main imports us for the
25    re-export).
26
27    Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
28    ``db_instance``,

```

```

16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     import uuid
30     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
31     from urllib.parse import quote
32
33     from backend import storage
34     from backend.config import (
35         PHASE_COMPILE,
36         PHASE_SEGMENT,
37         PHASE_VALIDATION,
38         PLACE_CLOUD_L4,
39         AppConfig,
40         StitchingConfig,
41         resolve_phase_placement,
42     )
43     from backend.pipeline.segmenters.base import segmenter_registry
44     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
45     from backend.pipeline.validators.base import ValidationResult, registry
46     from backend.results import Failure
47     from backend.utils.paths import output_base, safe_output_filename
48
49     if (
50         TYPE_CHECKING
51     ): # runtime import would be circular (main imports us for the re-export)
52         from backend.main import ProcessRequest # noqa: F401
53
54     logger = logging.getLogger(__name__)
55
56     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
57     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
58     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
59     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
60     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
61     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
62     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
63
64
65     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
66         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
67
68         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
69         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
70         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
71         """
72         if segments:

```

```

73         db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
74     else:
75         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
76
77
78     def _validation_failure_message(failures: List[Any]) -> str:
79         details = []
80         for failure in failures:
81             name = getattr(failure, "validator_name", None) or "validation"
82             message = (getattr(failure, "message", None) or "").strip()
83             details.append(f"{name}: {message}" if message else str(name))
84         if not details:
85             return "Video failed validation checks."
86         return "Video failed validation: " + "; ".join(details)
87
88
89     def _report_config_for_input(config: Any, raw_input: str) -> Any:
90         """Return report config that keeps remote-input reports out of disposable
91         scratch."""
92         storage_config = (
93             config.get("storage")
94             if isinstance(config, dict)
95             else getattr(config, "storage", None)
96         )
97         try:
98             if storage.input_is_local(raw_input, storage_config):
99                 return config
100         except ValueError:
101             return config
102         try:
103             cfg = (
104                 config.model_dump(mode="json")
105                 if hasattr(config, "model_dump")
106                 else dict(config or {})
107             )
108         except Exception:
109             cfg = {}
110         report = dict(cfg.get("report") or {})
111         if report.get("output_dir"):
112             return config
113         report["output_dir"] = os.path.join(output_base(config), "reports")
114         cfg["report"] = report
115         return cfg
116
117     def _ingest_remote_segments(
118         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
119     ) -> int:
120         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
121         the bucket and
122         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
123         call ? so
124         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
125         were produced.
126         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
127         pattern (cloud
128         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
129         malformed row is
130         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
131         temp dir is
132         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
133         a job failure
134         and prunes any partial rows (destroy-AFTER-confirm)."""
135         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
136         ref = storage.parse_uri(uri)

```

```

130     n = 0
131     with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
132         local = storage.fetch(
133             uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
134         )
135         with open(local, newline="", encoding="utf-8") as f:
136             for row in csv.DictReader(f):
137                 if n >= max_rows:
138                     logger.warning(
139                         "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
140                             max_rows,
141                             uri,
142                     )
143                     break
144                 try:
145                     start, end = float(row["start"]), float(row["end"])
146                 except (KeyError, TypeError, ValueError):
147                     continue
148                 if not (math.isfinite(start) and math.isfinite(end)):
149                     continue # reject inf/nan times rather than persist a junk rally
150                 meta: Dict[str, Any] = {
151                     "source": "cloud_run",
152                     "ending_reason": (row.get("ending_reason") or "").strip(),
153                 }
154                 sc = (row.get("shot_count") or "").strip()
155                 if sc:
156                     try:
157                         meta["shot_count"] = int(
158                             float(sc)
159                         ) # OverflowError on 'inf'/'1e400'
160                     except (ValueError, OverflowError):
161                         meta["shot_count"] = sc
162                 db.add_play_segment(video_id, start, end, metadata=meta)
163                 n += 1
164     return n
165
166
167     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
168     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
169     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
170     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
171     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
172     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})
173
174
175     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
176         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
177         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
178         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
179         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
180         A non-weights segmenter returns True (nothing local to check)."""
181         name = (seg_name or "").strip().lower()
182         if name not in _WEIGHTS_BACKED_SEGMENTERS:
183             return True

```

```

184     idx = getattr(cfg, "indexing", None)
185     substrate = (
186         getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
187     ).strip().lower()
188     if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
189         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
190     else: # wasb_hybrid, or fusion on the (default) WASB substrate
191         path = (
192             os.environ.get("WASB_WEIGHTS")
193             or getattr(getattr(idx, "wasb", None), "weights_path", "")
194             or ""
195         )
196     return bool(path) and os.path.exists(os.path.expanduser(path))
197
198
199 def _cloud_available(cfg) -> bool:
200     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
201 3).
202     True when the control plane is wired for Cloud Run: either the configured default
203     target is
204     already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
205     coordinates
206     (``storage.output_root``)
207     are present so a forced per-request override can actually dispatch + collect a
208     result.
209     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
210     would work."""
211     deployment = getattr(cfg, "deployment", None)
212     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
213         return True
214     has_job = bool(
215         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
216     )
217     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
218     return bool(has_job and out_root)
219
220
221 def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
222     """Would DETECT for this request run on the L4 worker (vs in-process)? Mirrors the
223     ``maybe_dispatch_remote`` decision WITHOUT side effects (#521), so
224     ``execute_processing`` can
225     decide whether to DELEGATE validation to the worker
226     (``phase_placement.validation``). A per-request
227     picker choice wins; otherwise ``phase_placement.segment`` (whose legacy default is
228     cloud iff
229     ``compute_target==cloud_run``) gates it, and cloud must actually be available. A
230     pre-dispatch reject
231     (e.g. cloud requested but unconfigured) counts as NOT-on-cloud ? nothing runs
232     remotely."""
233     choice = (req_compute or "").strip().lower()
234     if choice == "local":
235         return False
236     if choice == "cloud":
237         return _cloud_available(cfg)
238     return (
239         resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
240         and _cloud_available(cfg)
241     )
242
243
244 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:

```

```

237     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
238     minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
239     with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
240     (queue/cold-start), so the very first tick already shows movement."""
241     minutes = int(elapsed_sec) // 60
242     if minutes < 1:
243         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
244     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
245
246
247     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
248         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
249         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
250         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
251         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
252         if getattr(input_ref, "backend", "") != "gcs":
253             return None
254         sc = _storage_settings(cfg)
255         if not sc:
256             return None
257         try:
258             st = storage.get_storage("gcs", config=sc).stat(input_ref)
259             return getattr(st, "md5", None) or None
260         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
261             logger.info(
262                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
263                 getattr(input_ref, "uri", input_ref),
264                 exc_info=True,
265             )
266             return None
267
268
269     def _cached_process_payload(
270         video_id: str, validation_results, segments, requested_compute
271     ) -> Dict[str, Any]:
272         """The /api/process result for a video whose detection already exists ? ONE shape
for
273         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
274         return {
275             "video_id": video_id,
276             "status": "success",
277             "message": (
278                 f"Reused {len(segments)} cached play segments. "
279                 "Pass force=true to reprocess this video."
280             ),
281             "results": validation_results or [],
282             "play_segments": segments,
283             "cached": True,
284             "compute": {"path": "cached", "requested": requested_compute},
285         }
286
287
288     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
289         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
290         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
291         ref = storage.parse_uri(uri)
292         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
293             local = storage.fetch(

```

```

294         uri, os.path.join(td, ref.name or "report.json"), config=storage_config
295     )
296     with open(local, encoding="utf-8") as f:
297         loaded = json.load(f)
298     return loaded if isinstance(loaded, dict) else None
299
300
301     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
302     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
303     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
304     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
305     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
306     _REHYDRATE_LOCKS_GUARD = threading.Lock()
307
308
309     def _rehydrate_lock(video_id: str) -> threading.Lock:
310         with _REHYDRATE_LOCKS_GUARD:
311             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
312
313
314     def probe_process_cache(db, cfg, video_id: str) -> bool:
315         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
316
317         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
318         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
319         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
320         errors report False (? the normal dispatch path with its guards)."""
321         try:
322             existing = db.get_video(video_id)
323             if (
324                 existing
325                 and existing.get("status") == "processed"
326                 and db.query_play_segments(video_id, {})
327             ):
328                 return True
329             sc = _storage_settings(cfg)
330             if not sc or (sc.get("backend") or "local") == "local":
331                 return False
332             output_root = (sc.get("output_root") or "").rstrip("/")
333             if not output_root:
334                 return False
335             report_uri = f"{output_root}/outputs/{video_id}/report.json"
336             if not storage.exists(report_uri, config=sc):
337                 return False
338             report = _read_bucket_json(report_uri, sc)
339             return bool(report and report.get("status") == "success")
340         except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
341             logger.warning(
342                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
343             )
344             return False
345
346
347     def cached_process_result(
348         db, cfg, video_id: str, video_path: str, requested_compute=None
349     ) -> Optional[Dict[str, Any]]:
350         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
351         the DB first, then the bucket (``outputs/<md>/report.json`` with ``status=success``

```

```

352 re-ingests via ``_ingest_remote_segments``: **the DB is a cache, the bucket is the
353 source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
354 completed work found (or the check failed) ? the caller dispatches normally. The
bucket
355 branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
356 partial earlier ingest is replaced, never double-counted.
357
358 Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
359 per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
360 in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
361 job before invoking (writes happen post-claim)."""
362 try:
363     with _rehydrate_lock(video_id):
364         existing = db.get_video(video_id)
365         if existing and existing.get("status") == "processed":
366             segs = db.query_play_segments(video_id, {})
367             if segs:
368                 return _cached_process_payload(
369                     video_id,
370                     existing.get("validation_results", []),
371                     segs,
372                     requested_compute,
373                 )
374         sc = _storage_settings(cfg)
375         if not sc or (sc.get("backend") or "local") == "local":
376             return None
377         output_root = (sc.get("output_root") or "").rstrip("/")
378         if not output_root:
379             return None
380         outputs_uri = f"{output_root}/outputs/{video_id}"
381         report_uri = f"{outputs_uri}/report.json"
382         if not storage.exists(report_uri, config=sc):
383             return None
384         report = _read_bucket_json(report_uri, sc)
385         if not report or report.get("status") != "success":
386             return None
387         if not video_path:
388             # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
389             # carries the worker's --video-uri (P3); older reports fall back to the
390             # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
391             # recover, and the next same-md5 submit re-binds the truthful path via
392             # add_video's upsert.
393             video_path = str(report.get("video_uri") or "") or _first_object_under(
394                 f"{output_root}/videos/{video_id}/", sc
395             )
396         play_wm, cand_wm = db.segment_watermarks(video_id)
397         db.add_video(video_id, video_path, "processed", [])
398         n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
399         if n <= 0:
400             return None
401         segments = [
402             s
403             for s in db.query_play_segments(video_id, {})
404             if int(s.get("id", 0)) > play_wm
405         ]
406         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
407         if not segments:
408             return None
409         logger.info(
410             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time

```

```

cache hit)",
411             n,
412             video_id,
413             outputs_uri,
414         )
415         return _cached_process_payload(video_id, [], segments, requested_compute)
416     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
417         logger.warning(
418             "[API] submit-time cache check failed for %s ? dispatching normally",
419             video_id,
420             exc_info=True,
421         )
422         return None
423
424
425     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
426         """The first object URI under a bucket prefix, or "" ? best-effort source
427         recovery."""
428         try:
429             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
430         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
431             return ""
432
433     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
434
435
436     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
437         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
438         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
439         so
440         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
441         DB
442         has the video's rows now (re-query). Non-md5 ids return False without touching the
443         bucket; every failure degrades to False (the read just reports what the DB has)."""
444         vid = (video_id or "").strip().lower()
445         if not _MD5_HEX_RE.fullmatch(vid):
446             return False
447         if not probe_process_cache(db, cfg, vid):
448             return False
449         return cached_process_result(db, cfg, vid, "") is not None
450
451     def _maybe_dispatch_remote(
452         cfg,
453         db,
454         req: "ProcessRequest",
455         video_id: str,
456         seg_name: str,
457         val_results,
458         play_wm: int,
459         cand_wm: int,
460         notify,
461     ) -> Optional[Tuple[Dict[str, Any], bool]]:
462         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
463         ``deployment.compute_target
464         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
465         intervals into
466         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
467         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
468         the cloud job
469         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
470         observability report
471         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-

```

```

process segmenter
468     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
469     the in-process path stays byte-identical)."""
470     from backend.compute import ComputeJobSpec, get_compute_backend
471
472     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
473         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
474         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
475         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
476         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
477         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
478         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
479         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
480         return (
481             {
482                 "video_id": video_id,
483                 "status": "failed",
484                 "message": msg,
485                 "results": [r.model_dump() for r in val_results],
486                 "play_segments": [],
487                 "compute": {
488                     "path": "cloud_run" if ran else "not_run",
489                     "requested": req.compute,
490                     "target": "cloud_run",
491                     "dispatched": ran,
492                 },
493             },
494             ran,
495         )
496
497         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
499         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500         choice = (req.compute or "").strip().lower()
501         if choice and choice not in ("local", "cloud"):
502             logger.warning(
503                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
504                 req.compute,
505             )
506             choice = ""
507         if choice == "local":
508             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
509             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
510             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
511             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
512             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
513             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
514             # below exactly as before (the weights-present check is the escape hatch).
515             if (

```

```

516         (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
517         and _cloud_available(cfg)
518         and not _local_detector_weights_present(cfg, seg_name)
519     ):
520         return _failed(
521             f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
522             "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
523             False,
524         )
525         return None # explicit "run on my machine" ? in-process regardless of the
server's target
526     if choice == "cloud":
527         if not _cloud_available(cfg):
528             return _failed(
529                 "cloud-serving was requested but this server isn't configured for it "
530                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
531                 False,
532             )
533         compute = get_compute_backend(cfg, target_override="cloud_run")
534     else:
535         # No explicit picker ? consult phase_placement.segment (#521). Its legacy
default is
536         # cloud_run_l4 IFF compute_target==cloud_run, so this preserves today's
behaviour; setting
537         # phase_placement.segment relocates DETECT by config alone (e.g. force it back
on the
538         # control-plane, or onto the L4 on a default-local box wired for cloud).
539         if (
540             resolve_phase_placement(cfg, PHASE_SEGMENT) == PLACE_CLOUD_L4
541             and _cloud_available(cfg)
542         ):
543             compute = get_compute_backend(cfg, target_override="cloud_run")
544         else:
545             compute = None
546     if compute is None:
547         return None # default-OFF / placed on control_plane ? the in-process segmenter
below
548
549     storage_cfg = getattr(cfg, "storage", None)
550
551     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
552     try:
553         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
554     except ValueError as e:
555         return _failed(
556             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
557         )
558     if input_ref.backend == "local":
559         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
560         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
561         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
562         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
563         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
564         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
565         # default, while a bare/local-looking root can't stage no matter what

```

```

input_backend says
566         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
567         # gives us nothing to build the staged URI under).
568         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
569         try:
570             root_backend = (
571                 storage.parse_uri(input_root).backend if input_root else "local"
572             )
573         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
574             root_backend = "local"
575         if root_backend == "local":
576             return _failed(
577                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
578                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
579                 "Run job.",
580                 False,
581             )
582         local_src = input_ref.key # resolved local path of the upload
583         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
584         try:
585             notify("staging upload to bucket")
586             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
587             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
588             storage.put(
589                 local_src,
590                 staged_uri,
591                 config=(
592                     storage_cfg.model_dump() if storage_cfg is not None else None
593                 ),
594             )
595         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
596             logger.error(
597                 "[API] cloud-serving: failed to stage %s -> %s: %s",
598                 local_src,
599                 staged_uri,
600                 e,
601                 exc_info=True,
602             )
603             return _failed(
604                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
605                 False,
606             )
607         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
608         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
609         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
610         req.video_path = staged_uri
611         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
612         db.add_video(
613             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
614         )
615         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
616         if not out_root:
617             return _failed(

```

```

618         "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
619         "? that's where the Cloud Run job writes the result.",
620         False,
621     )
622
623     notify("dispatching (cloud_run)")
624     logger.info(
625         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626         req.video_path,
627         out_root,
628     )
629     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
630     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
631     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
632     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
633     overrides = []
634     idx_cfg = getattr(cfg, "indexing", None)
635     sk = getattr(idx_cfg, "skip_ai_handoff", None)
636     if sk is not None:
637         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
638     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
639     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
640     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
641     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
642     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
643     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
644     wasb_cfg = getattr(idx_cfg, "wasb", None)
645     if wasb_cfg is not None:
646         decode_backend = getattr(wasb_cfg, "decode_backend", None)
647         if decode_backend:
648             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
649         batch_size = getattr(wasb_cfg, "batch_size", None)
650         if batch_size:
651             overrides.append(f"indexing.wasb.batch_size={batch_size}")
652         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
653         if prefetch:
654             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
655     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
656     # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
657     # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
658     # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
659     # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
660     # then worker-rejected at its default 20.0 after a successful GPU dispatch).
661     # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
662     # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
663     # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the

```

```

664 # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ?
see
665 # _execute_processing), so the worker must NOT skip: it becomes the gate.
666 worker_skips_validation = (
667     resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
668 )
669 spec = ComputeJobSpec(
670     video_uri=req.video_path,
671     out_uri=out_root,
672     segmenter=seg_name,
673     config_overrides=tuple(overrides),
674     skip_validation=worker_skips_validation,
675 )
676 try:
677     # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
678     # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
679     # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
680     # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
681     result = compute.run_infer(
682         spec,
683         on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
684     )
685 except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
686     logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
687     return _failed(f"cloud dispatch error: {e}", True)
688 if result.returncode != 0:
689     return _failed(f"cloud job failed (returncode {result.returncode}).", True)
690 if result.video_id and result.video_id != video_id:
691     # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
692     # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
693     logger.warning(
694         "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
695         "and the worker hash; ingesting under the local id.",
696         result.video_id,
697         video_id,
698     )
699
700 notify("ingesting (cloud_run)")
701 storage_dict = (
702     storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
703     if hasattr(storage_cfg, "model_dump")
704     else {}
705 )
706 try:
707     _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
708 except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
709     logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
710     return _failed(
711         f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
712         True,
713     )
714
715 # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
716 segments = [
717     s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
718 ]

```

```

719     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
720     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
721     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
722     provenance = {
723         "path": "cloud_run",
724         "requested": req.compute,
725         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
726         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
727         "execution": result.execution,
728         "outputs_uri": result.outputs_uri,
729     }
730     logger.info(
731         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
732         result.execution,
733         provenance["job"],
734         provenance["region"],
735         result.outputs_uri,
736         video_id,
737     )
738     return (
739         {
740             "video_id": video_id,
741             "status": "success",
742             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
743             "results": [r.model_dump() for r in val_results],
744             "play_segments": segments,
745             "compute": provenance,
746         },
747         True,
748     )
749
750
751 def _execute_processing(
752     config=None, db=None, req=None, progress=None
753 ) -> Dict[str, Any]:
754     """The full hash ? validate ? segment ? report pipeline for one video.
755
756     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
757     module-level ``global_config``/``db_instance`` (backward-compatible with existing
758     tests that pass only ``req``). Route handlers always pass explicit params; older
759     test code that calls ``_execute_processing(req)`` still works.
760
761     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
762     Raises on unexpected internal errors ? the caller records those as a job failure
763     """
764     import backend.main as _main # call-time seam: main's globals/model stay patchable
765
766     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
767     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
768     if config is not None and not isinstance(config, AppConfig):
769         # Old signature: _execute_processing(req, progress=None)
770         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
771             req = db
772             db = None
773         elif req is None and isinstance(config, _main.ProcessRequest):
774             req = config
775             config = None
776     # Resolve config/db from module globals when not passed explicitly
777     if config is None:

```

```

778         config = _main.global_config
779     if db is None:
780         db = _main.db_instance
781
782     notify = progress or (lambda stage: None)
783
784     notify("hashing")
785     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
786     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
787     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
788     # consumers (hash / validators / segmenter) use the resolved local path.
789     local_video = ""
790     try:
791         local_video = storage.acquire_local_input(
792             req.video_path, getattr(config, "storage", None)
793         )
794         video_id = _main.compute_video_id(local_video)
795     except Exception:
796         storage.cleanup_acquired_local_input(
797             req.video_path, local_video, getattr(config, "storage", None)
798         )
799         raise
800     try:
801         existing_video = db.get_video(video_id)
802     except Exception:
803         storage.cleanup_acquired_local_input(
804             req.video_path, local_video, getattr(config, "storage", None)
805         )
806         raise
807     was_reingest = existing_video is not None
808
809     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
810     default_seg = getattr(
811         getattr(config, "indexing", None), "default_segmenter", "motion"
812     )
813     seg_name = req.segmenter or default_seg
814     segmenter_actual = None
815     segmentation_ran = False
816     val_results: List[Any] = []
817     val_context: Dict[str, Any] = {}
818     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
819     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
820     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
821     compute_actual: Optional[str] = None
822     response: Optional[Dict[str, Any]] = None
823     try:
824         if (
825             existing_video
826             and existing_video.get("status") == "processed"
827             and not req.force
828         ):
829             cached_segments = db.query_play_segments(video_id, {})
830             if cached_segments:
831                 logger.info(
832                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
833                     video_id,
834                 )
835             response = _cached_process_payload(

```

```

836         video_id,
837         existing_video.get("validation_results", []),
838         cached_segments,
839         getattr(req, "compute", None),
840     )
841     return response
842
843     # 1. Run pluggable validators (with-context variant surfaces fprobe_meta for
the report) ?
844     #     UNLESS this exact content already has a cached verdict under the current
validator config.
845     notify("validating")
846     # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
847     # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
848     # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
849     # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
850     # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
851     # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
852     # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
853     # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
854     # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
855     # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
856     # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
857     #
858     # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
859     # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
860     # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
861     # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
862     # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
863     # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
864     # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
segment).
865     validate_on_control_plane = (
866         resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
867         or not _will_detect_on_cloud(config, getattr(req, "compute", None))
868     )
869     if not validate_on_control_plane:
870         logger.info(
871             "[API] validation delegated to the L4 worker "
872             "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
873             "validator suite; the worker is the gate for %s.",
874             req.video_path,
875         )
876     val_fingerprint = (
877         registry.config_fingerprint(config) if validate_on_control_plane else ""
878     )
879     # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault

```

```

(unreadable row,
880         # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
881         # failing the job. The cache is a pure speedup ? never a new failure mode.
882         reused_validation = False
883         if validate_on_control_plane and not req.force:
884             try:
885                 cached = db.get_validation_cache(video_id)
886                 if cached is not None and cached.get("fingerprint") == val_fingerprint:
887                     # Replay the stored ValidationResult list; val_context stays {} (no
fresh
888                     # ffmpeg_meta ? the report's media block degrades gracefully to
size-only, the
889                     # deliberate cost of skipping the decode). Segmentation reads
neither of these.
890                     val_results = [
891                         ValidationResult(**r) for r in cached.get("results", [])
892                     ]
893                     reused_validation = True
894                     logger.info(
895                         "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
896                         "validator config unchanged ? skipping the validator suite. "
897                         "force=true re-validates.",
898                         cached.get("passed"),
899                         video_id,
900                     )
901             except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
902                 reused_validation = False
903                 logger.warning(
904                     "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
905                     video_id,
906                     exc_info=True,
907                 )
908             if validate_on_control_plane and not reused_validation:
909                 logger.info(f"Running validations for {req.video_path}...")
910                 val_results, val_context = registry.run_all_with_context(local_video,
config)
911             failures = [r for r in val_results if not r.passed]
912             if validate_on_control_plane and not reused_validation:
913                 # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
914                 # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
915                 db.set_validation_cache(
916                     video_id,
917                     val_fingerprint,
918                     not failures,
919                     [r.model_dump() for r in val_results],
920                 )
921
922             if failures:
923                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
924                 # status; it no longer destroys the video's prior good segments.
925                 db.add_video(
926                     video_id,
927                     req.video_path,
928                     "failed",
929                     [r.model_dump() for r in val_results],
930                 )
931             response = {
932                 "video_id": video_id,
933                 "status": "failed",

```

```

934         "message": _validation_failure_message(failures),
935         "results": [r.model_dump() for r in val_results],
936     }
937     return response
938
939 # 2. Run segmenter & indexer
940 logger.info("[API] Video passed checks. Segmenting and indexing...")
941 db.add_video(
942     video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
943 )
944
945 segmenter_cls = segmenter_registry.get(seg_name)
946 if not segmenter_cls:
947     # Submit-time validation already 400s unknown names; this is the defensive
948     # in-worker path (e.g. config default pointing at an unregistered
segmenter).
949     available = list(segmenter_registry.list_available().keys())
950     response = {
951         "video_id": video_id,
952         "status": "failed",
953         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
954         "results": [r.model_dump() for r in val_results],
955         "play_segments": [],
956     }
957     return response
958
959 logger.info(f"[API] Using segmenter: {seg_name}")
960 notify(f"segmenting ({seg_name})")
961 segmenter_actual = seg_name
962 # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
963 # if the run yields segments; otherwise drop the new partial rows and keep the
old.
964 play_wm, cand_wm = db.segment_watermarks(video_id)
965 # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
966 # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
967 # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
968 # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
969 # they're dropped on the cloud path in this slice ? a documented follow-up.)
970 remote = _maybe_dispatch_remote(
971     config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
972 )
973 if remote is not None:
974     response, segmentation_ran = (
975         remote # `ran` = did the cloud job actually execute (report)
976     )
977     return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
978 # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
979 compute_actual = "in_process"
980 segmenter = segmenter_cls(db, config)
981 try:
982     # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
983     # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
984     with _LOCAL_COMPUTE_LANE:
985         result = segmenter.process_video(
986             video_id, local_video, req.player_pool, req.max_frames

```

```

987         )
988     except Exception:
989         # A raising segmenter must not leave half-written rows: restore the pre-run
990         # state (same destroy-after-confirm contract as the Failure branch), then
991         let
992             # the job worker record the failure.
993             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
994             raise
995     segmentation_ran = True
996
997     if isinstance(result, Failure):
998         logger.error(f"[API] Segmenter failed: {result.message}")
999         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
1000         response = {
1001             "video_id": video_id,
1002             "status": "failed",
1003             "message": f"Segmenter '{seg_name}' failed: {result.message}",
1004             "results": [r.model_dump() for r in val_results],
1005             "play_segments": [],
1006         }
1007         return response
1008
1009     segments = result.value if hasattr(result, "value") else result
1010     notify("finalizing")
1011     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
1012
1013     response = {
1014         "video_id": video_id,
1015         "status": "success",
1016         "message": f"Successfully indexed {len(segments)} play segments using
1017         '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
1018         .value
1019         "results": [r.model_dump() for r in val_results],
1020         "play_segments": segments,
1021     }
1022     return response
1023
1024     finally:
1025         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
1026         already set
1027         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
1028         other path stamp
1029         # the actual path here so the job result PROVES which backend ran (in_process /
1030         not_run) and
1031         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
1032         then visible.
1033         if isinstance(response, dict):
1034             response.setdefault(
1035                 "compute",
1036                 {
1037                     "path": compute_actual or "not_run",
1038                     "requested": getattr(req, "compute", None),
1039                 },
1040             )
1041         prov = response.get("compute", {}) if isinstance(response, dict) else {}
1042         logger.info(
1043             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
1044             video_id,
1045             getattr(req, "compute", None),
1046             prov.get("path"),
1047             prov.get("dispatched"),
1048             (response or {}).get("status") if isinstance(response, dict) else None,
1049         )
1050         # Always emit the per-video observability report (next to the video, or to
1051         # report.output_dir). Never raises. Surface the paths in the response. (The
1052         compute-path

```

```

1044         # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
1045         # doesn't surface arbitrary run signals.)
1046         paths = _main.emit_indexer_report(
1047             db=db,
1048             video_id=video_id,
1049             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
1050             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
1051             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
1052             # still preserved in DB provenance via db.add_video.
1053             video_path=local_video,
1054             config=_report_config_for_input(config, req.video_path),
1055             val_results=val_results,
1056             val_context=val_context,
1057             segmenter_requested=req.segmenter,
1058             segmenter_actual=segmenter_actual,
1059             was_reingest=was_reingest,
1060             segmentation_ran=segmentation_ran,
1061         )
1062         if paths and isinstance(response, dict):
1063             response["reports"] = paths
1064         storage.cleanup_acquired_local_input(
1065             req.video_path, local_video, getattr(config, "storage", None)
1066         )
1067
1068
1069     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1070         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1071
1072         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
1073         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
1074         to a no-op rather than blocking a job it cannot measure."""
1075         try:
1076             sc = (
1077                 storage_config.model_dump()
1078                 if hasattr(storage_config, "model_dump")
1079                 else (storage_config or {})
1080             )
1081             backend = storage.get_storage(input_ref.backend, config=sc)
1082             return int(backend.stat(input_ref).size)
1083         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
1084             logger.debug(
1085                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
1086             )
1087             return None
1088
1089
1090     class _CompileError(Exception):
1091         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
1092         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
1093
1094         def __init__(self, status_code: int, detail: str):
1095             super().__init__(detail)
1096             self.status_code = status_code
1097             self.detail = detail
1098
1099

```

```

1100     def _resolve_compile(
1101         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1102     ):
1103         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
1104         resolve, since a
1105         row could change between submit and run). Verifies the video exists, the requested
1106         segments match
1107         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
1108         shot_count are exempt
1109         so manual/legacy picks can't silently vanish), and the output name is traversal-
1110         safe. Raises
1111         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
1112         safe_out_name)``."""
1113         video = db.get_video(video_id)
1114         if not video:
1115             raise _CompileError(404, "Indexed video record not found.")
1116         # Reject path traversal / absolute names (os.path.join would otherwise write
1117         anywhere on disk).
1118         try:
1119             out_name = safe_output_filename(output_filename)
1120         except ValueError as e:
1121             raise _CompileError(400, str(e)) from e
1122
1123         all_segments = db.query_play_segments(video_id, {})
1124         matched = [s for s in all_segments if s["id"] in segment_ids]
1125         if not matched:
1126             raise _CompileError(400, "No matching play segments found to stitch.")
1127
1128         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1129         kept = []
1130         for s in matched:
1131             meta = s.get("metadata") or {}
1132             sc = meta.get("shot_count") if isinstance(meta, dict) else None
1133             try:
1134                 if sc is not None and int(sc) < stitching.min_shot_count:
1135                     continue
1136             except (TypeError, ValueError):
1137                 pass # unparseable count -> treat as unknown, keep
1138             kept.append(s)
1139         if len(kept) < len(matched):
1140             logger.info(
1141                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
1142                 floor.",
1143                 stitching.min_shot_count,
1144                 len(matched) - len(kept),
1145                 len(matched),
1146             )
1147         if not kept:
1148             raise _CompileError(
1149                 400,
1150                 f"All {len(matched)} matching segments fall below "
1151                 f"stitching.min_shot_count={stitching.min_shot_count}.",
1152             )
1153         return video, kept, out_name
1154
1155     def reel_object_uri(cfg, video_id: str, name: str) -> str:
1156         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1157         (#463), or "" when
1158         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1159         compile
1160         mirror-upload and the /api/highlights redirect:
1161         ``<output_root>/highlights/<video_id>/<name>``."""
1162         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1163         if not root or "://" not in root:

```

```

1155         return ""
1156         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1157
1158
1159     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1160         """The active storage settings dict (per-backend config for the storage layer ? S3
1161         endpoint/region,
1162         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1163         default backend
1164         isn't silently constructed as a fresh default."""
1165         sc = getattr(cfg, "storage", None)
1166         return sc.model_dump() if sc is not None else None
1167
1168     def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1169         if rendition != "sd":
1170             return None
1171         renditions = (
1172             stitching.get("renditions")
1173             if isinstance(stitching, dict)
1174             else getattr(stitching, "renditions", None)
1175         )
1176         if isinstance(renditions, dict):
1177             return renditions.get("sd")
1178         return getattr(renditions, "sd", None)
1179
1180     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1181         """A short-lived signed URL for a reel mirrored to the output bucket, so
1182         /api/highlights can
1183         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
1184         ephemeral local
1185         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
1186         config for BOTH
1187         the existence check and the signing (extracted here to keep backend/main.py lean ?
1188         P23/P24)."""
1189         reel_uri = reel_object_uri(cfg, video_id, name)
1190         if not reel_uri:
1191             return None
1192         settings = _storage_settings(cfg)
1193         if storage.exists(reel_uri, config=settings):
1194             return storage.signed_url(reel_uri, config=settings, method="GET")
1195         return None

```

5. assistant (2026-07-08T06:16:39.651Z)

Let me read the rest of orchestration.py, particularly the compile paths.

6. user (2026-07-08T06:16:41.241Z)

```

1194     def _compile_wait_message(elapsed_sec: float) -> str:
1195         """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1196         ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
1197         minutes = int(elapsed_sec) // 60
1198         if minutes < 1:
1199             return "stitching on cloud GPU (NVENC) ? starting up"
1200         return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1201
1202
1203     def _stitch_reel(
1204         cfg,
1205         stitching,
1206         video_id: str,

```

```

1207     source_ref: str,
1208     time_pairs: List[Tuple[float, float]],
1209     out_name: str,
1210     locale: Optional[str],
1211     notify,
1212     rendition: str = "original",
1213 ) -> Dict[str, Any]:
1214     """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1215     (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1216     needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1217     with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1218     trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1219     returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1220     ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure). The
1221     optional ``rendition`` selects the output encode profile; ``sd`` scales/transcodes
cheaply and
1222     never builds the original-size reel for that job."""
1223     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1224
1225     rendition = rendition or "original"
1226     if rendition not in {"original", "sd"}:
1227         return {
1228             "status": "failed",
1229             "message": f"Unsupported compile rendition: {rendition}",
1230             "video_id": video_id,
1231         }
1232     output_dir = (
1233         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1234     )
1235     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1236     # unknown/absent locale keeps the configured default unchanged.
1237     localized_watermark = localize_watermark(stitching.watermark, locale)
1238     encode_profile = _rendition_encode_profile(stitching, rendition)
1239     compiler = _main.VideoCompiler(
1240         output_dir,
1241         begin_padding=stitching.begin_padding,
1242         end_padding=stitching.end_padding,
1243         watermark=localized_watermark,
1244         encoder=getattr(stitching, "encoder", "libx264"),
1245         encode_profile=encode_profile,
1246     )
1247
1248     notify("stitching")
1249     logger.info(
1250         "[compile] stitching video_id=%s rendition=%s output=%s",
1251         video_id,
1252         rendition,
1253         out_name,
1254     )
1255     local_src = ""
1256     try:
1257         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1258         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1259         # same seam the process path uses); without it, compiling a bucket-ingested

```

```

video can't open gs://.
1260         local_src = storage.acquire_local_input(
1261             source_ref, getattr(cfg, "storage", None)
1262         )
1263         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1264         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1265         with _LOCAL_COMPUTE_LANE:
1266             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1267         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1268             logger.error(
1269                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1270             )
1271             return {
1272                 "status": "failed",
1273                 "message": f"Highlight stitching failed: {e}",
1274                 "video_id": video_id,
1275             }
1276         finally:
1277             if local_src:
1278                 storage.cleanup_acquired_local_input(
1279                     source_ref, local_src, getattr(cfg, "storage", None)
1280                 )
1281
1282         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1283         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1284         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1285         # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1286         # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1287         reel_uri = reel_object_uri(cfg, video_id, out_name)
1288         if reel_uri:
1289             try:
1290                 notify("uploading reel")
1291                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1292                 # project/client) is used, not a fresh default.
1293                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1294             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1295                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1296
1297         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1298         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1299         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1300         notify("finished")
1301         return {
1302             "status": "success",
1303             "filepath": out_path,
1304             "download_url": download_url,
1305             "video_id": video_id,
1306             "reel_uri": reel_uri or "",
1307             "filename": out_name,
1308             "rendition": rendition,
1309         }
1310
1311
1312     def _dispatch_compile_remote(

```

```

1313     cfg,
1314     db,
1315     video_id: str,
1316     segment_ids: List[int],
1317     output_filename: str,
1318     locale: Optional[str],
1319     notify,
1320     rendition: str = "original",
1321 ) -> Dict[str, Any]:
1322     """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1323     control-plane owns the DB, so it RESOLVES the reel's segments here
(`_resolve_compile`), stages a
1324     small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1325     to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1326     done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
1327     worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1328     ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1329     from backend.compute import CompileJobSpec, get_compute_backend
1330
1331     notify("resolving")
1332     try:
1333         video, kept, out_name = _resolve_compile(
1334             db, cfg, video_id, segment_ids, output_filename
1335         )
1336     except _CompileError as e:
1337         return {"status": "failed", "message": e.detail, "video_id": video_id}
1338
1339     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1340     storage_cfg = getattr(cfg, "storage", None)
1341     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1342     compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
1343     if compute is None:
1344         # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
1345         # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
1346         # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
1347         logger.warning(
1348             "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
1349             "stitching in-process on the control-plane instead.",
1350             out_root,
1351         )
1352         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1353         return _stitch_reel(
1354             cfg,
1355             stitching,
1356             video_id,
1357             video["filepath"],
1358             time_pairs,
1359             out_name,
1360             locale,
1361             notify,
1362             rendition,
1363         )
1364
1365     # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
1366     # the stitching config is the control-plane's RESOLVED one, with the encoder swapped

```

```

to the L4's
1367     # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
1368     stitch_dump = stitching.model_dump()
1369     stitch_dump["encoder"] = getattr(
1370         getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
1371     )
1372     spec_payload = {
1373         "video_id": video_id,
1374         "video_uri": video["filepath"],
1375         "output_name": out_name,
1376         "rendition": rendition,
1377         "locale": locale,
1378         "time_pairs": [
1379             [float(s["start_time"]), float(s["end_time"])] for s in kept
1380         ],
1381         "stitching": stitch_dump,
1382     }
1383     token = uuid.uuid4().hex
1384     spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
1385     try:
1386         notify("staging compile spec")
1387         with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
1388             local_spec = os.path.join(td, "spec.json")
1389             with open(local_spec, "w", encoding="utf-8") as f:
1390                 json.dump(spec_payload, f)
1391             storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
1392     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
1393         logger.error(
1394             "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
1395         )
1396     return {
1397         "status": "failed",
1398         "message": f"cloud compile: could not stage the compile spec: {e}",
1399         "video_id": video_id,
1400     }
1401
1402     # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1403     # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1404     overrides = [f"storage.output_root={out_root}"]
1405     backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1406     if backend_name:
1407         overrides.append(f"storage.backend={backend_name}")
1408     spec = CompileJobSpec(
1409         out_uri=out_root,
1410         spec_uri=spec_uri,
1411         video_id=video_id,
1412         output_name=out_name,
1413         config_overrides=tuple(overrides),
1414     )
1415
1416     notify("dispatching stitch (cloud_run)")
1417     logger.info(
1418         "[compile] dispatching stitch for video_id=%s rendition=%s -> Cloud Run Job
(out=%s, %d clip(s))",
1419         video_id,
1420         rendition,
1421         out_root,
1422         len(kept),
1423     )
1424     try:

```

```

1425         result = compute.run_compile(
1426             spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1427         )
1428     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1429         logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1430         return {
1431             "status": "failed",
1432             "message": f"cloud compile dispatch error: {e}",
1433             "video_id": video_id,
1434         }
1435     if result.returncode != 0:
1436         msg = str(
1437             result.report.get("message")
1438             or f"cloud compile job failed (returncode {result.returncode})."
1439         )
1440         return {"status": "failed", "message": msg, "video_id": video_id}
1441
1442     download_url = str(
1443         result.report.get("download_url")
1444         or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1445     )
1446     logger.info(
1447         "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s rendition=%s",
1448         result.execution,
1449         result.outputs_uri,
1450         video_id,
1451         rendition,
1452     )
1453     notify("finished")
1454     return {
1455         "status": "success",
1456         "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
1457         "download_url": download_url,
1458         "video_id": video_id,
1459         "reel_uri": str(result.report.get("reel_uri") or ""),
1460         "filename": str(result.report.get("filename") or out_name),
1461         "rendition": str(result.report.get("rendition") or rendition),
1462         "compute": {
1463             "path": "cloud_run",
1464             "execution": result.execution,
1465             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
1466             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
1467         },
1468     }
1469
1470
1471 def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1472     """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1473     the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
1474     result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
1475     a clean verdict instead of a worker-crash error.
1476
1477     #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
1478     dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1479     (``dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
1480     2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
1481     notify = progress or (lambda stage: None)
1482     params = json.loads(job.get("params") or "{}")

```

```

1483     video_id = job.get("video_id") or ""
1484     segment_ids = params.get("segment_ids") or []
1485     locale = params.get("locale")
1486     output_filename = params.get("output_filename", "")
1487     rendition = str(params.get("rendition") or "original")
1488     if rendition not in {"original", "sd"}:
1489         return {
1490             "status": "failed",
1491             "message": f"Unsupported compile rendition: {rendition}",
1492             "video_id": video_id,
1493         }
1494
1495     if (
1496         resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1497         and _cloud_available(cfg)
1498     ):
1499         return _dispatch_compile_remote(
1500             cfg, db, video_id, segment_ids, output_filename, locale, notify, rendition
1501         )
1502
1503     notify("resolving")
1504     try:
1505         video, kept, out_name = _resolve_compile(
1506             db, cfg, video_id, segment_ids, output_filename
1507         )
1508     except _CompileError as e:
1509         return {"status": "failed", "message": e.detail, "video_id": video_id}
1510
1511     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1512     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1513     return _stitch_reel(
1514         cfg,
1515         stitching,
1516         video_id,
1517         video["filepath"],
1518         time_pairs,
1519         out_name,
1520         locale,
1521         notify,
1522         rendition,
1523     )
1524

```

7. assistant (2026-07-08T06:17:19.176Z)

Now let me read the config models (validation guards), the compute backend factory, and the worker.

8. user (2026-07-08T06:17:23.309Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]

```

```

16
17
18 class ValidationRelevanceConfig(BaseModel):
19     court_green_lower: list[int] = Field(default=[35, 40, 40])
20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24 class ValidationConfig(BaseModel):
25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     pts_max_gap_seconds: float = Field(
31         default=10.0,
32         ge=0.0,
33         description=(
34             "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35             "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36             "a gap >10s suggests edited/missing content. Mobile phones, screen "
37             "recordings, and other encoders may use different GOP sizes ? tune "
38             "this threshold for your recording device."
39         ),
40     )
41     relevance: ValidationRelevanceConfig = Field(
42         default_factory=ValidationRelevanceConfig
43     )
44
45
46 class TrackNetConfig(BaseModel):
47     wsl_distro: str = "Ubuntu"
48     repo_dir: str = "~/models/TrackNetV4"
49     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50     conda_env: str = "TrackNetV4"
51     weights_path: str = ""
52     queue_length: int = 5
53     stage_in_wsl: bool = True
54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"

```

```

79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
    ---
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101    # 0 = no extra floor (the headroom guard still applies).
102    min_free_gb: float = 0.0
103    # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104    # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105    # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106    # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107    # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108    # Tri-state:
109    # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110    # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111    # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112    stream_video: Optional[bool] = None
113    # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114    # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115    # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116    # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117    # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118    batch_size: Optional[int] = None
119    # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120    # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121    prefetch_batches: Optional[int] = None
122    # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123    # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124    # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive

```

```

F.interpolate
125      # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126      # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127      # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128      # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129      # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130      # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131      decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133      @model_validator(mode="after")
134      def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135          # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136          # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137          # healthcheck hard-fails if auto resolves to cpu with this backend on.
138          if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139              raise ValueError(
140                  f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                  f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142              )
143          return self
144
145      @field_validator("batch_size")
146      @classmethod
147      def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148          # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149          # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150          # when truthy, so 0 stays "off/parity-default").
151          if v is not None and v < 1:
152              raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153          return v
154
155      @field_validator("prefetch_batches")
156      @classmethod
157      def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158          # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159          # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160          # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161          if v is not None and v < 0:
162              raise ValueError(
163                  f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
164              )
165          return v
166
167
168      class FusionConfig(BaseModel):
169          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
170
171          Every default reproduces control behaviour: the fusion segmenter persists the
172          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
173          true ? the person-cue features, but it does NOT gate the served handoff unless a
174          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court

```

```

mask
175         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
176         supplied ? off-court persons (spectators / adjacent courts) are excluded.
177         ""
178
179         # Which trajectory substrate seeds the windows (shuttle stays primary).
180         substrate: Literal["wasb", "tracknet"] = "wasb"
181         # Windowing velocity threshold for the fusion family (the engine reads
182         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
183         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
184         velocity_threshold: float = 0.02
185         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
186         # enabled ? so the default path never imports torch / touches the GPU.
187         cue_flags: dict[str, bool] = Field(default_factory=dict)
188         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
189         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
190         min_crossings: int = Field(default=0, ge=0)
191         # Served Gate B: when the person cue is on, drop windows with median in-court
players
192         # < this. 0 = OFF.
193         min_players: int = Field(default=0, ge=0)
194         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
195         court_polygon: Optional[list[list[float]]] = None
196         # Net line for the person two-sided-motion split (person features only ? the shuttle
197         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
198         net_axis: Literal["x", "y"] = "y"
199         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
200
201
202     class IndexingConfig(BaseModel):
203         default_segmenter: str = "yolo_hybrid"
204         use_gpu: bool = True
205         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210         detector_impl: str = "auto"
211         motion_threshold: float = 0.08
212         min_segment_duration: float = 3.0
213         dead_time_merge_gap: float = 2.0
214         chunk_duration: float = 90.0
215         chunk_overlap: float = 10.0
216         detector_model: str = "fasterrcnn_resnet50_fpn"
217         detector_score_threshold: float = 0.5
218         yolo_velocity_threshold: float = 0.15
219         yolo_frame_skip: int = 3
220         yolo_tracker: str = "bytetrack.yaml"
221         yolo_prefetch_workers: int = 2
222         min_action_window_duration: float = 2.0
223         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
225         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
226         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
227         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
228         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,

```

```

p=0.0005; cap=6
229      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
230      max_window_duration: float = 6.0
231      window_min_split_gap: float = (
232          0.5 # min internal gap (s) that qualifies as a split point
233      )
234      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
235      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
236      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
237      # Only cuts (recall can't drop). OFF by default until measured/promoted.
238      window_hard_cap: bool = False
239      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
245      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246      window_inactivity_end: float = 1.5
247      inactivity_rally_thresh: float = 0.20
248      inactivity_min_density: float = 0.30
249      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255      # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
256      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258      inactivity_temporal_density: bool = True
259      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
260      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).

```

```

267     window_blob_fallback: bool = False
268     # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269     # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270     # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
271     # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
272     window_blob_factor: float = 4.0
273     log_yolo_candidates: bool = True
274     store_yolo_candidates: bool = True
275     ai_handoff_padding: float = 2.0
276     ai_max_workers: int = 5
277     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278     # chunks of a high-bitrate (4K) source blow past the provider upload cap
279     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282     # Matches gemini_refine's --clip-scale-width default.
283     ai_chunk_scale_width: int = 1280
284     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289     skip_ai_handoff: bool = False
290     # Optional debug knob: trim every video to the first N seconds before processing.
291     debug_duration: Optional[float] = None
292
293     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295     fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297     @field_validator("default_segmenter")
298     @classmethod
299     def segmenter_must_be_registered(cls, v: str) -> str:
300         # Lazy import: the segmenter registry imports modules that may import
301         # config, so we resolve it at validation time rather than at module load.
302         from backend.pipeline.segmenters.base import segmenter_registry
303
304         if segmenter_registry.get(v) is None:
305             available = sorted(segmenter_registry.list_available().keys())
306             raise ValueError(
307                 f"Segmenter '{v}' is not registered. Available: {'', '.join(available)}"
308             )
309         return v
310
311     @model_validator(mode="after")
312     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314         # step makes `while t < duration: t += step` loop forever, growing memory before
any
315         # GPU work. Reject the bad config at load time instead.
316         if self.chunk_overlap >= self.chunk_duration:
317             raise ValueError(
318                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                 f"({self.chunk_duration}); otherwise chunking never advances."
320             )

```

```

321         return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
329         output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
341         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans" # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         ) # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True # faint backing box behind the text for legibility
348         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
349         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350             "bottom-right"
351         )
352
353
354     class SdRenditionConfig(BaseModel):
355         """Optional low-resolution reel encode profile.
356
357         This is always available to callers that explicitly request it; the knobs stay
358         configurable because different hosts may prefer a smaller 360p file or a more
359         readable 480p file. The original rendition remains the historical source-size
360         encode path and is intentionally not configurable through this profile.
361         """
362
363         max_width: int = Field(default=854, ge=16)
364         max_height: int = Field(default=480, ge=16)
365         crf: int = Field(default=30, ge=0, le=51)
366         audio_bitrate: str = "96k"
367         video_preset: str = "superfast"
368         pix_fmt: str = "yuv420p"
369
370
371     class StitchingRenditionsConfig(BaseModel):
372         sd: SdRenditionConfig = Field(default_factory=SdRenditionConfig)
373
374
375     class StitchingConfig(BaseModel):
376         begin_padding: float = 0.5
377         end_padding: float = 0.5
378         use_cache: bool = False
379         min_shot_count: int = 1
380         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
381         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
382         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
383         # fully-local no-AI path, whereas duration is derived from the rally's own

```

```

start/end. The
384     # local detector over-fires and its false positives are mostly SHORT (motion blips,
385     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
386     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
387     min_rally_duration: float = Field(default=0.0, ge=0.0)
388     # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
389     # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
390     # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-
encode that
391     # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested
encoder ONCE and
392     # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never
breaks. The
393     # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for
the L4 run;
394     # the control-plane's own local stitch keeps libx264.
395     encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
396     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
397     renditions: StitchingRenditionsConfig = Field(
398         default_factory=StitchingRenditionsConfig
399     )
400
401
402     class LoggingConfig(BaseModel):
403         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
404
405         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
406         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
407         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
408         # them to WARNING; set true to restore full chatter for request-flow debugging.
409         verbose_http: bool = False
410
411
412     class SecurityConfig(BaseModel):
413         # When set, /api/process video_path must resolve under this directory (hosted/tenant
414         # mode). Default None preserves the single-user local 'any local file' workflow.
415         allowed_video_root: Optional[str] = None
416
417         # --- Chunked upload (B2, PR-2) ---
418         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
419         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
420         # contain upload_dir or /api/process will reject the uploaded paths.
421         upload_dir: str = "output/uploads"
422         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
423         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
424         max_upload_mb: int = 4096
425         max_part_mb: int = 95
426         allowed_upload_extensions: List[str] = ["mp4", "mov"]
427
428         # --- M9 edge auth (Cloudflare Access, D9) ---
429         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
430         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
431         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
432         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
433         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
434         # application's audience tag. NO secrets here ? these are public identifiers.

```

```

435     edge_auth_enabled: bool = False
436     cf_team_domain: str = (
437         "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
438     )
439     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
440     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
441     # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
442     # OFF ? Cf-Access is a header, not a cookie.)
443
444
445     class StorageConfig(BaseModel):
446         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
447         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
448         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
449         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
450         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
451         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
452
453         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
454         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
455         output_root: str = ""
456         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
457         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
458         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
459         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
460         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
461         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
462         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
463         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
464         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
465         input_root: str = ""
466         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
467         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
468         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
469         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
470         # the legacy flat layout is used unchanged.
471         single_folder_per_video: bool = False
472         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
473         workspace_root: str = ""
474         # Cloud namespace prefix under the root so per-video folders sit tidily beside
475         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
476         workspace_prefix: str = "workspaces"
477         local: dict[str, Any] = Field(default_factory=dict)
478         s3: dict[str, Any] = Field(default_factory=dict)
479         gcs: dict[str, Any] = Field(default_factory=dict)
480         gdrive: dict[str, Any] = Field(default_factory=dict)
481         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the

```

```

hyphen?underscore lookup in
482     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
483     gdrive_api: dict[str, Any] = Field(default_factory=dict)
484
485
486     class PhasePlacementConfig(BaseModel):
487         """Per-phase machine placement (#521): move validation / segment(detect) /
compile(stitch)
488         between the control-plane and the L4 Cloud Run worker **by config, no code change**.
489
490         Each field is ``None`` (? the legacy default for that phase ? see
491         ``backend/config/placement.py``) or an explicit ``"control_plane"`` /
``"cloud_run_l4"``.
492         ``control_plane`` runs the phase in-process on the CPU-only control-plane;
``cloud_run_l4``
493         dispatches it to the GPU/NVENC worker Job. Unset ? byte-identical to pre-#521
behaviour.
494         Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read
these directly."""
495
496         validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
497         segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
498         compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
499
500
501     class DeploymentConfig(BaseModel):
502         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
503         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
504
505         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
506         behaves exactly as before this section existed. A profile is opt-in (``config.json``
507         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
508         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
509         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
510         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
511
512         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
513
514         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
515         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
516         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
517         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
518         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
519         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
520         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
521         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
522         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
523         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
524         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
525         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
526         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays

```

```

the
527         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
528         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
529         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
530         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
531         # Per-phase machine placement (#521). Unset ? today's placement (validation +
compile on the
532         # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
533         # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read
the sub-fields
534         # directly ? the resolver folds in the legacy compute_target default).
535         phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
536         # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for
the L4 stitch
537         # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
538         # set ``libx264`` to run the L4 stitch on CPU. Only consulted when compile is placed
on the L4;
539         # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
540         cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
541
542         @model_validator(mode="after")
543         def _validation_placement_needs_cloud_segment(self) -> "DeploymentConfig":
544             # A clip can only be validated ON the worker if the worker also DETECTS it there
? the worker
545             # is only launched for the segment(detect) phase. So validation=cloud_run_l4
with segment on
546             # the control-plane would leave NOBODY validating (the control-plane skips, and
no worker runs)
547             # ? reject that footgun at config load rather than ship a silently-unvalidated
clip.
548             if self.phase_placement.validation == "cloud_run_l4":
549                 seg = self.phase_placement.segment or (
550                     "cloud_run_l4" if self.compute_target == "cloud_run" else
"control_plane"
551                 )
552             if seg != "cloud_run_l4":
553                 raise ValueError(
554                     "deployment.phase_placement.validation='cloud_run_l4' requires
segment to also "
555                     "run on the L4 ? the worker can only validate a clip it also
detects. Set "
556                     "phase_placement.segment='cloud_run_l4' (or
compute_target='cloud_run'), or keep "
557                     "validation on the control_plane."
558                 )
559             return self
560
561
562     class BillingConfig(BaseModel):
563         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
564
565         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
566         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
567         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
568         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner

```

```

569         inserts a real, calibrated version and points ``pricebook_version`` at it.
570
571     ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
572     wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
573     pricebook + the recording seam; usage emission (the metering hooks) is a separate
step. """
574
575     enabled: bool = False
576     pricebook_version: str = (
577         "placeholder-v0" # the seeded $0 version; owner flips to a real one
578     )
579     fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
580     margin: float = 0.0 # resale markup (?0); 0 = bill at cost
581
582
583     class FlywheelConfig(BaseModel):
584         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
585         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
586         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
587         the owned model climbs toward the D11 ship floor.
588
589         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
590         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
591         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
592         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-row).
593         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
594         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
595         owner/counsel/GPU scope. Pairs with M9-consent. """
596
597         enabled: bool = False
598         workspace_prefix: str = (
599             "flywheel" # per-video workspace prefix for captured artifacts
600         )
601         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
602         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
603
604
605     class AppConfig(BaseModel):
606         """Root configuration object.
607
608         All runtime configuration should be accessed via an instance of this class
609         (or the loader) rather than raw dict lookups.
610
611
612         sport: Sport = "badminton"
613         ai_provider: str = "gemini"
614         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
615         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
616         ai_model: str = "gemini-flash-latest"
617
618         validation: ValidationConfig = Field(default_factory=ValidationConfig)
619         indexing: IndexingConfig = Field(default_factory=IndexingConfig)

```

```

620     output: OutputConfig = Field(default_factory=OutputConfig)
621     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
622     security: SecurityConfig = Field(default_factory=SecurityConfig)
623     logging: LoggingConfig = Field(default_factory=LoggingConfig)
624     storage: StorageConfig = Field(default_factory=StorageConfig)
625     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
626     billing: BillingConfig = Field(default_factory=BillingConfig)
627     flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
628
629     # Reject unknown keys ? typed config migration is complete (D3).
630     model_config = {
631         "extra": "forbid",
632         "validate_assignment": True,
633     }
634
635
636     # Convenience type alias for callers that want to annotate
637     Config = AppConfig
638

```

9. user (2026-07-08T06:17:23.362Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local:`` / ``s3:`` / ``gcs:`` is an identity passthrough; ``s3:`` / ``gcs:`` is an object store), so existing
9     local runs
10    are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
11    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
12    key) ?
13    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
14
15    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
16    CLI uses
17    (validator registry ? segmenter ? report), never a forked pipeline.
18
19    """
20
21    from __future__ import annotations
22
23    import argparse
24    import csv
25    import json
26    import logging
27    import os
28    import re
29    import sys
30    import tempfile
31    from typing import Any, List
32
33    from backend import storage
34    from backend.compute import ComputeJobSpec, get_compute_backend
35    from backend.config import (
36        StartupCheckError,
37        enforce_startup_checks,
38        load_config,
39        probe_cuda_available,
40    )
41
42    from backend.infrastructure.database import Database
43    from backend.pipeline.segmenters.base import segmenter_registry
44    from backend.pipeline.validators.base import registry as validator_registry

```

```

40     from backend.reporting import emit_indexer_report
41     from backend.results import Failure
42     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("backend.worker")
46
47
48     def _coerce(v: str) -> Any:
49         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50         str."""
51         low = v.lower()
52         if low in ("true", "false"):
53             return low == "true"
54         for cast in (int, float):
55             try:
56                 return cast(v)
57             except ValueError:
58                 pass
59         return v
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63         indexing.skip_ai_handoff=true)."""
64         for kv in sets or []:
65             key, sep, val = kv.partition("=")
66             if not sep:
67                 raise ValueError(f"--set expects k=v, got {kv!r}")
68             parts = key.split(".")
69             obj = config
70             for p in parts[:-1]:
71                 obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76         friendly
77         artifact (same schema as the rally-annotator labels)."""
78         with open(path, "w", newline="") as f:
79             w = csv.writer(f)
80             w.writerow(["start", "end", "shot_count", "ending_reason"])
81             for s in segments:
82                 w.writerow(
83                     [
84                         f"{float(s.get('start_time', 0.0)):.3f}",
85                         f"{float(s.get('end_time', 0.0)):.3f}",
86                         s.get("shot_count", ""),
87                         s.get("ending_reason", s.get("shot_count_confidence", "")),
88                     ]
89                 )
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row

```

```

102         # with its truthful path ? without this, only the segments recover.
103         "video_uri": video_uri,
104         "segments": [
105             {
106                 "start": float(s.get("start_time", 0.0)),
107                 "end": float(s.get("end_time", 0.0)),
108             }
109             for s in segments
110         ],
111     },
112     f,
113     indent=2,
114 )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily

```

```

imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure."""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode

```

```

215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:
237                 json.dump(marker, f)
238             storage.put(local, done_uri, config=storage_cfg)
239             logger.info("[worker] wrote completion marker -> %s", done_uri)
240         except Exception as e: # never fail a good run because the marker push hiccuped
241             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244     def cmd_infer(args: argparse.Namespace) -> int:
245         config = load_config(args.config) if args.config else load_config()
246         _apply_overrides(config, args.set)
247         if not _run_startup_checks(config):
248             return 2
249
250         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254         compute = get_compute_backend(config)
255         if compute is not None:
256             return _dispatch_remote(compute, args)
257
258         storage_cfg = config.storage.model_dump()
259
260         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261         os.makedirs(scratch, exist_ok=True)
262         config.output.output_dir = scratch
263
264         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265         local_video = storage.fetch(
266             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267         )
268         print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271         video_id = compute_video_id(local_video)

```

```

272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275     # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276     # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277     # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278     # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281     # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282     if getattr(args, "skip_validation", False):
283         print(
284             "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285             "gate and already validated this input."
286         )
287         val_results, val_context = [], {}
288     else:
289         val_results, val_context = validator_registry.run_all_with_context(
290             local_video, config
291         )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309         try:
310             _serialize_and_push_outputs(
311                 scratch,
312                 args.out_uri,
313                 video_id,
314                 [],
315                 "failed",
316                 storage_cfg,
317                 str(getattr(args, "video_uri", "")) or "",
318             )
319         except Exception as e: # never mask the real failure on an artifact-push hiccup
320             logger.warning("[worker] could not push failure artifacts: %s", e)

```

```

321         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322         if getattr(args, "done_uri", None):
323             _write_done_marker(
324                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325             )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339         seg_name = args.segmenter or config.indexing.default_segmenter
340         segmenter_cls = segmenter_registry.get(seg_name)
341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360         if not segments:
361             detector_impl = (
362                 os.environ.get("RALLY_DETECTOR_IMPL")
363                 or getattr(config.indexing, "detector_impl", None)
364                 or "auto"
365             )
366             logger.warning(
367                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373                 "detections, or the input resolution differs from the tuned proxy
resolution. "
374                 "Re-run with -v for the native command + frame counts.",
375                 video_id,
376                 segmenter_actual,

```

```

377         detector_impl,
378     )
379     finally:
380         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381         emit_indexer_report(
382             db=db,
383             video_id=video_id,
384             video_path=local_video,
385             config=config,
386             val_results=val_results,
387             val_context=val_context,
388             segmenter_requested=args.segmenter,
389             segmenter_actual=segmenter_actual,
390             was_reingest=False,
391             segmentation_ran=segmentation_ran,
392         )
393
394     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395     pushed = _serialize_and_push_outputs(
396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
artifact(s):"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415     if getattr(args, "done_uri", None):
416         _write_done_marker(
417             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418         )
419     return 0
420
421
422     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
`time_pairs`, the
424         resolved `stitching` config, the source `video_uri`, `video_id`,
`output_name` + `locale`.
425         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
426         local = storage.fetch(
427             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
428         )
429         with open(local, encoding="utf-8") as f:
430             data = json.load(f)
431         if not isinstance(data, dict):
432             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")

```

```

433         return data
434
435
436     def _write_compile_marker(
437         done_uri: str,
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "filename": result.get("filename", ""),
454                 "rendition": result.get("rendition", ""),
455                 "message": result.get("message", ""),
456             }
457             local = os.path.join(scratch, "_compile_done.json")
458             with open(local, "w", encoding="utf-8") as f:
459                 json.dump(marker, f)
460             storage.put(local, done_uri, config=storage_cfg)
461             logger.info("[worker] wrote compile marker -> %s", done_uri)
462         except Exception as e: # never fail a good stitch because the marker push hiccuped
463             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
464
465
466     def cmd_compile(args: argparse.Namespace) -> int:
467         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
468         2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
469         the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
470         the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
471         forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
472         bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
473         from backend import orchestration
474         from backend.config import StitchingConfig
475
476         config = load_config(args.config) if args.config else load_config()
477         _apply_overrides(config, args.set)
478         storage_cfg = config.storage.model_dump()
479
480         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
481         os.makedirs(scratch, exist_ok=True)
482         # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
483         config.output.output_dir = scratch
484         # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
485         # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket

```

```

486     # destination even if the override were dropped.
487     if not (config.storage.output_root or "").strip():
488         config.storage.output_root = args.out_uri
489
490     spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
491     video_id = str(spec.get("video_id") or "")
492     out_name = str(spec.get("output_name") or "")
493     rendition = str(spec.get("rendition") or "original")
494     source_ref = str(spec.get("video_uri") or "")
495     locale = spec.get("locale")
496     time_pairs = [
497         (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
498     ]
499     stitching = StitchingConfig(**(spec.get("stitching") or {}))
500
501     print(
502         f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
503         f"encoder={stitching.encoder} rendition={rendition} out={out_name}"
504     )
505
506     result = orchestration._stitch_reel(
507         config,
508         stitching,
509         video_id,
510         source_ref,
511         time_pairs,
512         out_name,
513         locale,
514         lambda stage: print(f"[worker] compile: {stage}"),
515         rendition,
516     )
517     status = str(result.get("status") or "failed")
518     rc = 0 if status == "success" else 3
519     if status != "success":
520         print(f"[worker] compile FAILED: {result.get('message')}")
521
522     # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
523     # passes it.
524     if getattr(args, "done_uri", None):
525         _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
526     return rc
527
528     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
529     #: so a
530     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
531     #: collision).
532     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
533
534     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
535     #: DD>_<name>.rallies.csv`,
536     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
537     #: but is NOT
538     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
539     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
540
541     def _label_clip_name(fname: str) -> str:
542         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
543         filename.
544
545         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
546         prefix, so a
547         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

```

```

544 * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
stamped)
545 * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
stem)
546 * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
547 * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
548
549 Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
550 video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS $7b #1).
551 """
552 name = fname.replace(".rallies.csv", "").replace(".csv", "")
553 return _LABEL_DATE_PREFIX.sub("", name)
554
555
556 def _probe_video_fps_width(path: str):
557     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
558
559     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
560 fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
561     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
562     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
563     probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
564     rather than guesses.
565     """
566     import json
567     import subprocess
568
569     try:
570         out = subprocess.check_output(
571             [
572                 ffprobe_bin(),
573                 "-v",
574                 "error",
575                 "-select_streams",
576                 "v:0",
577                 "-show_entries",
578                 "stream=r_frame_rate,width",
579                 "-of",
580                 "json",
581                 path,
582             ],
583             stderr=subprocess.DEVNULL,
584         ).decode()
585         st = (json.loads(out).get("streams") or [{}])[0]
586         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
587         fps = float(num) / float(den or "1")
588         width = float(st.get("width") or 0)
589         if fps > 0 and width > 0:
590             return fps, width
591     except (
592         subprocess.SubprocessError,
593         ValueError,
594         KeyError,
595         OSError,
596         json.JSONDecodeError,
597     ):

```

```

598         pass
599     return None
600
601
602 def _resolve_train_detector(args: argparse.Namespace, config: Any):
603     """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
604     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
605     box,
606     stub=CPU mock). Returns the runner, or prints an error + returns None.
607
608     Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
609     the
610     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
611     has
612     no shuttle detector and is rejected.
613     """
614     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
615
616     seg_cls = segmenter_registry.get(args.segmenter)
617     if not seg_cls:
618         print(
619             f"[worker] unknown segmenter: {args.segmenter!r} "
620             f"(have {list(segmenter_registry.list_available())})"
621         )
622         return None
623     seg = seg_cls(None, config)
624     if not isinstance(seg, TrajectoryHybridSegmenter):
625         print(
626             f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
627             f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
628         )
629         return None
630     try:
631         runner, _ = seg._resolve_runner(config.indexing)
632     except NotImplementedError as e:
633         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
634         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
635         traceback.
636         print(f"[worker] detector not available: {e}")
637         return None
638     ok, msg = runner.healthcheck()
639     if not ok:
640         print(f"[worker] detector environment not ready: {msg}")
641         return None
642     print(f"[worker] detector ready ({args.segmenter}): {msg}")
643     return runner
644
645
646 def cmd_train(args: argparse.Namespace) -> int:
647     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
648     shell out
649     to training.gen0.harness (backend must NOT import training) -> push the artifact
650     bundle.
651
652     Two trajectory sources (the train pipeline + harness are identical either way):
653     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
654     synthetic
655         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
656         CI.
657     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
658         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
659     This
660         is the M3.5 "first real training" path; only the trajectory source flips.
661     """
662     import subprocess

```

```

654
655     config = load_config(args.config) if args.config else load_config()
656     _apply_overrides(config, args.set)
657     if not _run_startup_checks(config):
658         return 2
659     storage_cfg = config.storage.model_dump()
660
661     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
662     labels_dir = os.path.join(scratch, "labels")
663     traj_dir = os.path.join(scratch, "trajectories")
664     videos_dir = os.path.join(scratch, "videos")
665     os.makedirs(labels_dir, exist_ok=True)
666     os.makedirs(traj_dir, exist_ok=True)
667
668     corpus = args.corpus_uri.rstrip("/")
669     label_uris = sorted(
670         u
671         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
672         if u.lower().endswith(".csv")
673     )
674     if len(label_uris) < 2:
675         print(
676             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
677             f"under {corpus}/labels/"
678         )
679         return 2
680
681     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
682     runner = None
683     video_by_stem: dict = {}
684     if args.trajectory_source == "detector":
685         os.makedirs(videos_dir, exist_ok=True)
686         runner = _resolve_train_detector(args, config)
687         if runner is None:
688             return 2
689         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
690             vname = storage.parse_uri(vuri).name
691             if not vname.lower().endswith(_VIDEO_EXTS):
692                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
693             video_by_stem[os.path.splitext(vname)[0]] = vuri
694
695     print(f"[worker] trajectory source: {args.trajectory_source}")
696     specs: List[dict] = []
697     for uri in label_uris:
698         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
699         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
700         local_label = storage.fetch(
701             uri, os.path.join(labels_dir, fname), config=storage_cfg
702         )
703         if args.trajectory_source == "detector":
704             vuri = video_by_stem.get(name)
705             if not vuri:
706                 print(
707                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
708                 )
709                 continue
710             local_video = storage.fetch(
711                 vuri,
712                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
713                 config=storage_cfg,
714             )
715             video_id = compute_video_id(local_video)

```

```

716         # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
717         # wrong fps silently mis-aligns every label, so skip the video if ffmpeg
can't read it.
718         meta = _probe_video_fps_width(local_video)
719         if meta is None:
720             print(
721                 f"[worker] could not probe fps/width for {name!r} (ffmpeg) ?
skipping (won't guess).")
722             )
723             continue
724         fps, frame_width = meta
725         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
726         if not traj:
727             print(
728                 f"[worker] detector produced no trajectory for {name!r} ? skipping."
729             )
730             continue
731         specs.append(
732             {
733                 "name": name,
734                 "trajectory": traj,
735                 "labels": local_label,
736                 "fps": fps,
737                 "frame_width": frame_width,
738             }
739         )
740     else:
741         from backend.pipeline.detectors.stub_runner import (
742             synth_trajectory_from_labels,
743         )
744
745         traj = os.path.join(traj_dir, f"{name}_traj.csv")
746         synth_trajectory_from_labels(
747             local_label, traj, fps=args.fps, frame_width=args.frame_width
748         )
749         specs.append(
750             {
751                 "name": name,
752                 "trajectory": traj,
753                 "labels": local_label,
754                 "fps": args.fps,
755                 "frame_width": args.frame_width,
756             }
757         )
758
759     if len(specs) < 2:
760         print(
761             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
762         )
763         return 2
764     manifest_path = os.path.join(scratch, "manifest.json")
765     with open(manifest_path, "w", encoding="utf-8") as f:
766         json.dump(specs, f, indent=2)
767     src_label = (
768         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
769     )
770     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
771
772     out_dir = os.path.join(scratch, "artifacts")
773     cmd = [
774         sys.executable,
775         "-m",
776         "training.gen0.harness",

```

```

777         "--manifest",
778         manifest_path,
779         "--out",
780         out_dir,
781         "--version",
782         args.version,
783     ]
784     if args.floor:
785         floor_local = (
786             storage.fetch(
787                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
788             )
789             if "://" in args.floor
790             else args.floor
791         )
792         cmd += ["--floor", floor_local]
793     print("[worker] training:", " ".join(cmd))
794     rc = subprocess.run(cmd).returncode
795     if rc != 0:
796         print(f"[worker] gen0 harness failed (rc={rc})")
797         return rc
798
799     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
800     bundle = os.path.join(out_dir, args.version)
801     out_prefix = args.out_uri.rstrip("/")
802     pushed = []
803     for fn in sorted(os.listdir(bundle)):
804         uri = f"{out_prefix}/weights/{args.version}/{fn}"
805         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
806         pushed.append(uri)
807     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
808     for u in pushed:
809         print("    ", u)
810     return 0
811
812
813 def build_parser() -> argparse.ArgumentParser:
814     p = argparse.ArgumentParser(
815         prog="python -m backend.worker",
816         description="Storage-aware worker: pull ? run pipeline ? push.",
817     )
818     p.add_argument(
819         "-v",
820         "--verbose",
821         action="store_true",
822         help="DEBUG logging (the native detector command, frame counts, per-stage detail)",
823     )
824     sub = p.add_subparsers(dest="cmd", required=True)
825
826     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
827     pi.add_argument(
828         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
829     )
830     pi.add_argument(
831         "--out-uri",
832         required=True,
833         help="output prefix (outputs/<video_id>/? is appended)",
834     )
835     pi.add_argument(
836         "--segmenter", default=None, help="default: indexing.default_segmenter"
837     )
838     pi.add_argument(
839         "--config", default=None, help="config.json path (default: ./config.json)"
840     )

```

```

841     pi.add_argument(
842         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
843     )
844     pi.add_argument("--max-frames", type=int, default=None)
845     pi.add_argument(
846         "--done-uri",
847         default=None,
848         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
849         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
850         "Default-OFF: omit it for the normal local run.",
851     )
852     pi.add_argument(
853         "--set",
854         action="append",
855         default=[],
856         metavar="k=v",
857         help="config override, e.g. --set indexing.skip_ai_handoff=true",
858     )
859     pi.add_argument(
860         "--skip-validation",
861         action="store_true",
862         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
863         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
864         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
865         "`worker infer` CLI still validates.",
866     )
867     pi.set_defaults(func=cmd_infer)
868
869     pc = sub.add_parser(
870         "compile",
871         help="#521: stitch a highlight reel on this worker (L4 NVENC) from a staged
compile-spec",
872     )
873     pc.add_argument(
874         "--spec-uri",
875         required=True,
876         help="storage URI of the control-plane-staged compile-spec JSON (time-pairs +
stitching "
877         "config + source uri + video_id)",
878     )
879     pc.add_argument(
880         "--out-uri",
881         default="",
882         help="output ROOT (gs://? bucket); the reel is mirrored to
highlights/<video_id>/<name>. "
883         "Also forwarded via --set storage.output_root; either resolves the reel path.",
884     )
885     pc.add_argument(
886         "--done-uri",
887         default=None,
888         help="storage URI for the compile completion MARKER (rc + download_url), so a
remote "
889         "dispatcher's bucket-poll terminates. Default-OFF: omit for a direct-CLI
stitch.",
890     )
891     pc.add_argument(
892         "--config", default=None, help="config.json path (default: ./config.json)"
893     )
894     pc.add_argument(
895         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
896     )
897     pc.add_argument(

```

```

898         "--set",
899         action="append",
900         default=[],
901         metavar="k=v",
902         help="config override, e.g. --set storage.output_root=gs://bucket",
903     )
904     pc.set_defaults(func=cmd_compile)
905
906     pt = sub.add_parser(
907         "train",
908         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
909     )
910     pt.add_argument(
911         "--corpus-uri",
912         required=True,
913         help="prefix containing labels/ (+ videos/ for "
914         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
915     )
916     pt.add_argument(
917         "--out-uri",
918         required=True,
919         help="output prefix (weights/<version>/ is appended)",
920     )
921     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
922     pt.add_argument(
923         "--trajectory-source",
924         choices=["synth", "detector"],
925         default="synth",
926         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
927         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
928         "(needs videos/ in the corpus; honours detector_impl).",
929     )
930     pt.add_argument(
931         "--segmenter",
932         default="wasb_hybrid",
933         help="trajectory hybrid whose detector is used for --trajectory-source",
934     )
935     pt.add_argument(
936         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
937     )
938     pt.add_argument("--config", default=None)
939     pt.add_argument("--scratch", default=None)
940     pt.add_argument(
941         "--fps",
942         type=float,
943         default=30.0,
944         help="synth trajectory fps (detector probes the video)",
945     )
946     pt.add_argument(
947         "--frame-width",
948         type=float,
949         default=1280.0,
950         help="synth trajectory frame width (detector probes the video)",
951     )
952     pt.add_argument("--set", action="append", default=[], metavar="k=v")
953     pt.set_defaults(func=cmd_train)
954     return p
955
956
957 def _setup_logging(verbose: bool) -> None:
958     """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
959     diagnostics are
960     VISIBLE ? the worker previously ran with no logging config, so every
961     segmenter/detector log

```

```

960         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
961         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
962         entry), not at import, so importing the module never reconfigures a caller's
logging.""
963         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
964         # configured logging at import), which would leave the level at WARNING and re-
suppress the
965         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
966         logging.basicConfig(
967             level=logging.DEBUG if verbose else logging.INFO,
968             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
969             datefmt="%H:%M:%S",
970             force=True,
971         )
972
973
974     def main(argv=None) -> int:
975         args = build_parser().parse_args(argv)
976         _setup_logging(getattr(args, "verbose", False))
977         return args.func(args)
978
979
980     if __name__ == "__main__":
981         sys.exit(main())
982

```

10. assistant (2026-07-08T06:18:22.991Z)

Let me read the compute backend to understand dispatch semantics and whether
`get\_compute\_backend` gates on anything.

11. user (2026-07-08T06:18:25.115Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8     """
9
10    from __future__ import annotations
11
12    import dataclasses
13    import os
14    from typing import Any, Optional
15
16    from backend.compute.base import (
17        CompileJobSpec,
18        ComputeBackend,
19        ComputeJobSpec,
20        ComputeResult,
21        RemoteExecutionFailed,
22    )
23    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
24

```

```

25     __all__ = [
26         "ComputeBackend",
27         "ComputeJobSpec",
28         "CompileJobSpec",
29         "ComputeResult",
30         "RemoteExecutionFailed",
31         "get_compute_backend",
32     ]
33
34
35     def _storage_dict(config: Any) -> dict:
36         storage = getattr(config, "storage", None)
37         if storage is None:
38             return {}
39         if hasattr(storage, "model_dump"):
40             return storage.model_dump() # type: ignore[no-any-return]
41         return dict(storage) if isinstance(storage, dict) else {}
42
43
44     def get_compute_backend(
45         config: Any,
46         *,
47         run_client: Optional[Any] = None,
48         policy: ExecutionPolicy = DEFAULT_POLICY,
49         token: Optional[str] = None,
50         target_override: Optional[str] = None,
51     ) -> Optional[ComputeBackend]:
52         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
53         to run
54         in-process (the default-OFF path).
55
56         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
57         ``CloudRunCompute``). Its
58         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
59         ``CLOUD_RUN_REGION``
60         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
61         injectable (tests
62         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
63         verified on M7).
64
65         ``target_override`` lets a single request force a target (the SPA compute-picker
66         maps a
67         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
68         the
69         server's configured default. ``None`` falls back to
70         ``config.deployment.compute_target``.
71
72         """
73         target = (
74             target_override
75             if target_override is not None
76             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
77         )
78         if target != "cloud_run":
79             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
80
81         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
82         selected.
83         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
84
85         # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
86         # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout
87         so a
88         # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the
89         old
90         # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps

```

```

an
79     # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
80     if policy is DEFAULT_POLICY:
81         max_wait = getattr(
82             getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
83         )
84         if max_wait:
85             policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))
86
87     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
88     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
89     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
90     client = run_client if run_client is not None else GoogleCloudRunJobClient()
91     return CloudRunCompute(
92         run_client=client,
93         job_name=job_name,
94         region=region,
95         project=project,
96         storage_config=_storage_dict(config),
97         policy=policy,
98         token=token,
99     )
100

```

12. user (2026-07-08T06:18:25.384Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11        job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12        worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13        the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        *Fast negative path:* while the marker stays the SOLE success signal, each poll
also probes the
15        Cloud Run Executions API ? if the execution has TERMINATED without ever writing a
marker (a
16        worker crash/OOM/argparse-abort before the marker push), the shim fails fast +
diagnostically
17        (``RemoteExecutionFailed``) instead of spinning the full ~65-min budget on a futile
wait.
18    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
19        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
20
21    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
22    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
23

```

```

24     The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
25     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
26     owner-verified on the M7 real run.
27     """
28
29     from __future__ import annotations
30
31     import json
32     import logging
33     import os
34     import tempfile
35     import uuid
36     from abc import ABC, abstractmethod
37     from typing import Any, Callable, List, Optional
38
39     from backend.compute.base import (
40         CompileJobSpec,
41         ComputeBackend,
42         ComputeJobSpec,
43         ComputeResult,
44         RemoteExecutionFailed,
45     )
46     from backend.infrastructure.execution_policy import (
47         DEFAULT_POLICY,
48         ExecutionPolicy,
49         PolicyTimeout,
50         poll_until,
51     )
52     from backend.storage import fetch, get_storage, parse_uri
53
54     LOG = logging.getLogger("compute.cloud_run")
55
56     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
57     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
58     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
59         "deployment.compute_target=local_gpu",
60         "indexing.detector_impl=native",
61     )
62
63
64     class CloudRunJobClient(ABC):
65         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
66
67         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
68         overrides the job's container ``args`` for this execution and starts it."""
69
70         @abstractmethod
71         def run_job(
72             self,
73             job_name: str,
74             argv: List[str],
75             *,
76             region: str,
77             project: Optional[str] = None,
78         ) -> str:
79             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
80             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

```

```

81
82     @abstractmethod
83     def cancel_execution(self, execution: str) -> None:
84         """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
85         times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
86         May raise ? the caller treats every failure as best-effort (the original poll
timeout is
87         NEVER masked by a cancel error)."""
88
89     def execution_terminal_state(self, execution: str) -> Optional[str]:
90         """Return the execution's TERMINAL state as a short label (``"Failed"`` /
``"Cancelled"`` /
91         ``"Succeeded"``) if it has finished, else ``None`` (still pending/running, or
the state
92         can't be determined).
93
94         ADVISORY, not authoritative: it feeds ``run_infer``'s fast negative path so a
worker that
95         terminated WITHOUT writing the done-marker (crash/OOM/pre-marker abort) fails
fast instead
96         of bucket-polling the full budget. The done-marker stays the ONLY success
signal.
97
98         NON-abstract with a default of ``None`` (additive / default-OFF): a client that
can't ? or
99         chooses not to ? poll the Executions API degrades transparently to today's
marker-only
100        polling. Implementations MAY raise on a transient API error ? ``run_infer``
treats a raise
101        as ``None`` (unknown ? keep waiting), so a flaky status API never fails a
healthy run."""
102        return None
103
104
105    class GoogleCloudRunJobClient(CloudRunJobClient):
106        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
107        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
108
109        def run_job(
110            self,
111            job_name: str,
112            argv: List[str],
113            *,
114            region: str,
115            project: Optional[str] = None,
116        ) -> str:
117            # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
118            # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
119            # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
120            if not project:
121                raise RuntimeError(
122                    "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
123                    "to resolve the job path; set it on the control plane (see
get_compute_backend)."
124                )
125            try:
126                from google.cloud import run_v2 # type: ignore[attr-defined]

```

```

127         except Exception as exc: # pragma: no cover - real SDK not present in CI
128             raise RuntimeError(
129                 "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
130                 "control plane (it is intentionally NOT a CI/test dependency).")
131             ) from exc
132         client = (
133             run_v2.JobsClient()
134         ) # pragma: no cover - exercised only on the real M7 run
135         name = client.job_path(project, region, job_name)
136         overrides = run_v2.RunJobRequest.Overrides(
137             container_overrides=[
138                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
139             ]
140         )
141         op = client.run_job(
142             request=run_v2.RunJobRequest(name=name, overrides=overrides)
143         )
144         return (
145             getattr(op, "metadata", None)
146             and getattr(op.metadata, "name", "")
147             or "execution"
148         )
149
150     def cancel_execution(self, execution: str) -> None:
151         # run_job falls back to the literal string "execution" when the operation
metadata carries
152         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
153         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
154         # google-cloud-run is absent).
155         if "/executions/" not in (execution or ""):
156             raise RuntimeError(
157                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
158                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
159                 "one from the operation metadata, so there is nothing to cancel by
name).")
160         )
161         try:
162             from google.cloud import run_v2 # type: ignore[attr-defined]
163         except Exception as exc: # pragma: no cover - real SDK not present in CI
164             raise RuntimeError(
165                 "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
166                 "control plane (it is intentionally NOT a CI/test dependency).")
167             ) from exc
168         client = (
169             run_v2.ExecutionsClient()
170         ) # pragma: no cover - exercised only on a real run
171         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
172
173     def execution_terminal_state(self, execution: str) -> Optional[str]:
174         # Advisory status probe (see the ABC docstring): fetch the execution and
classify it via the
175         # pure, unit-tested ``_classify_execution_state``. A non-resolvable name /
absent SDK returns
176         # None (can't probe ? keep marker-polling) ? unlike run_job/cancel this never
raises on bad
177         # input, because the probe is best-effort and must not fail a healthy run.
178         if "/executions/" not in (execution or ""):
179             return None
180         try:

```

```

181         from google.cloud import run_v2 # type: ignore[attr-defined]
182     except Exception: # pragma: no cover - real SDK not present in CI
183         return None
184     client = (
185         run_v2.ExecutionsClient()
186     ) # pragma: no cover - exercised only on a real run
187     exe = client.get_execution( # pragma: no cover - real run only
188         request=run_v2.GetExecutionRequest(name=execution)
189     )
190     return _classify_execution_state(exe) # pragma: no cover - real run only
191
192
193     def _classify_execution_state(exe: Any) -> Optional[str]:
194         """Map a Cloud Run ``run_v2.Execution`` to a terminal state label (``"Failed"`` /
195         ``"Cancelled"``
196         / ``"Succeeded"``), or ``None`` if it is still pending/running. PURE (no
197         SDK/network) so it is
198         unit-tested against both real proto-plus ``Execution`` objects and lightweight fakes
199         ? the actual
200         ``get_execution`` fetch is the only part that can't run in CI.
201         ``google-cloud-run`` is a **proto-plus** library, so a ``google.protobuf.Timestamp``
202         field is
203         marshalled to a native Python ``datetime`` when set and to ``None`` when unset ? NOT
204         to a proto
205         ``Timestamp`` with a ``.seconds`` field (a ``datetime`` has ``.second``, singular).
206         Terminality is
207         therefore ``completion_time`` is not None; the per-task counts (plain ints) then
208         classify it,
209         with a cancel taking precedence (a cancelled execution reports ``cancelled_count >
210         0``)."""
211         if getattr(exe, "completion_time", None) is None:
212             return None # no terminal completion timestamp yet ? still pending/running
213         if getattr(exe, "cancelled_count", 0):
214             return "Cancelled"
215         if getattr(exe, "failed_count", 0):
216             return "Failed"
217         if getattr(exe, "succeeded_count", 0):
218             return "Succeeded"
219         return "Failed" # terminal but no positive success ? treat as failed (conservative)
220
221
222     class CloudRunCompute(ComputeBackend):
223         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
224
225         def __init__(
226             self,
227             *,
228             run_client: CloudRunJobClient,
229             job_name: str,
230             region: str,
231             project: Optional[str] = None,
232             storage_config: Optional[dict] = None,
233             policy: ExecutionPolicy = DEFAULT_POLICY,
234             token: Optional[str] = None,
235             container_overrides: Optional[tuple[str, ...]] = None,
236         ) -> None:
237             self._client = run_client
238             self._job_name = job_name
239             self._region = region
240             self._project = project
241             self._storage_config = storage_config or {}
242             self._policy = policy
243             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
244             collide.

```

```

237         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
238         self._token = token
239         self._container_overrides = (
240             _DEFAULT_CONTAINER_OVERRIDES
241             if container_overrides is None
242             else tuple(container_overrides)
243         )
244
245     def run_infer(
246         self,
247         spec: ComputeJobSpec,
248         *,
249         on_wait: "Callable[[float], None] | None" = None,
250     ) -> ComputeResult:
251         out_root = spec.out_uri.rstrip("/")
252         token = self._token or uuid.uuid4().hex
253         done_uri = f"{out_root}/_dispatch/{token}.done"
254         argv: List[str] = [
255             "infer",
256             "--video-uri",
257             spec.video_uri,
258             "--out-uri",
259             spec.out_uri,
260             "--done-uri",
261             done_uri,
262         ]
263         if spec.segmenter:
264             argv += ["--segmenter", spec.segmenter]
265         if spec.skip_validation:
266             # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
267             # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
268             argv.append("--skip-validation")
269         for kv in (*spec.config_overrides, *self._container_overrides):
270             argv += ["--set", kv]
271
272         execution = self._client.run_job(
273             self._job_name, argv, region=self._region, project=self._project
274         )
275         LOG.info(
276             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
277             self._job_name,
278             execution,
279             done_uri,
280         )
281
282         marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
infer")
283         video_id = str(marker.get("video_id", ""))
284         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
285         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
286         rc = int(marker.get("returncode", 1))
287         if not video_id:
288             LOG.warning(
289                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
290                 "video_id) ? treating as failure",
291                 self._job_name,
292             )
293         rc = rc or 1
294         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
295         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}

```

```

296         LOG.info(
297             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
298         )
299         return ComputeResult(
300             returncode=rc,
301             outputs_uri=outputs_uri,
302             video_id=video_id,
303             report=report,
304             execution=str(execution or ""),
305         )
306
307     def run_compile(
308         self,
309         spec: "CompileJobSpec",
310         *,
311         on_wait: "Callable[[float], None] | None" = None,
312     ) -> ComputeResult:
313         """Dispatch a reel-COMPILE (#521): run ``python -m backend.worker compile`` on
the L4 (NVENC),
314         bucket-poll the done-marker, and return the worker's rc + marker. Reuses the
EXACT
315         dispatch?poll?marker machinery as ``run_infer`` (``await_marker`` ? crash-fast
+ timeout
316         rescue), so a stitch that crashes or overruns fails the same way a detection
does ? not a
317         ~65-min opaque hang."""
318         out_root = spec.out_uri.rstrip("/")
319         token = self._token or uuid.uuid4().hex
320         # Distinct marker prefix from infer's _dispatch/ so a compile + a detection for
the same
321         # bucket can never collide on a marker.
322         done_uri = f"{out_root}/_compile/{token}.done"
323         argv: List[str] = [
324             "compile",
325             "--out-uri",
326             spec.out_uri,
327             "--spec-uri",
328             spec.spec_uri,
329             "--done-uri",
330             done_uri,
331         ]
332         # Compile carries NO container-inline overrides: cmd_compile never consults
compute_target and
333         # never re-dispatches, so the only overrides are the CP-forwarded
storage/encoder knobs.
334         for kv in spec.config_overrides:
335             argv += ["--set", kv]
336
337         execution = self._client.run_job(
338             self._job_name, argv, region=self._region, project=self._project
339         )
340         LOG.info(
341             "cloud-run: dispatched COMPILE job=%s exec=%s; bucket-polling %s",
342             self._job_name,
343             execution,
344             done_uri,
345         )
346         marker = self._await_marker(done_uri, execution, on_wait, label="cloud-run-
compile")
347         video_id = str(marker.get("video_id", "")) or spec.video_id
348         rc = int(marker.get("returncode", 1))
349         LOG.info(
350             "cloud-run: COMPILE job=%s done (video_id=%s rc=%d)",
351             self._job_name,
352             video_id,

```

```

353         rc,
354     )
355     # The marker carries the worker's stitch result (download_url / reel_uri /
status); surface it
356     # as ``report`` so the dispatcher returns the same {status, filepath,
download_url} shape the
357     # SPA expects. ``outputs_uri`` is the mirrored reel URI when present.
358     return ComputeResult(
359         returncode=rc,
360         outputs_uri=str(marker.get("reel_uri", "") or ""),
361         video_id=video_id,
362         report=marker,
363         execution=str(execution or ""),
364     )
365
366     def _await_marker(
367         self,
368         done_uri: str,
369         execution: object,
370         on_wait: "Callable[[float], None] | None",
371         *,
372         label: str,
373     ) -> dict:
374         """Bucket-poll ``done_uri`` to a completion marker ? shared by ``run_infer`` and
375         ``run_compile``. ``on_wait`` is the #482 live heartbeat (each still-waiting tick
reaches the
376         caller so job.progress moves). A ``RemoteExecutionFailed`` (the fast negative
path in
377         ``_poll_check``) is NOT a ``PolicyTimeout`` so it propagates uncaught ? the
dispatch handler
378         maps it to a distinct failure; a genuine poll TIMEOUT is handled (rescue-or-
cancel)."""
379         try:
380             return poll_until(
381                 lambda: self._poll_check(done_uri, str(execution or "")),
382                 self._policy,
383                 label=label,
384                 logger=LOG,
385                 on_tick=on_wait,
386             )
387         except PolicyTimeout as timeout_exc:
388             return self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
389
390     def _poll_check(self, done_uri: str, execution: str) -> Optional[dict]:
391         """One bucket-poll iteration, hardened with an Executions-API fast negative
path.
392
393         The done-marker is the SOLE success signal and is checked FIRST ? a present
marker always
394         wins (even if the execution's status later flips), so a worker that wrote a
SUCCESS *or a
395         FAILURE* marker resolves through the normal path with its real returncode. Only
when the
396         marker is absent do we consult the execution status: a worker that TERMINATED
without ever
397         writing a marker (crash / OOM / an old image argparse-aborting on an unknown
flag) would
398         otherwise make this poll spin the full ``remote_poll_max_wait_sec`` (~65 min)
before failing
399         opaquely. Detecting that terminal state collapses the whole class to a prompt,
actionable
400         failure.
401
402         Returns the marker dict (resolve the poll), ``None`` (keep waiting), or raises
403         ``RemoteExecutionFailed`` (abort the poll immediately ? the execution is over,

```

```

no marker
404         will ever appear)."""
405         marker = self._read_marker(done_uri)
406         if marker is not None:
407             return marker # success signal ? authoritative even if status later flips
408         state = self._terminal_execution_state(execution)
409         if state is None:
410             return None # still pending/running, or status unknown ? keep bucket-
polling
411         # The execution is terminal but no marker is present. Re-check the marker ONCE
to close the
412         # race where the worker wrote it between our two reads (mirrors the post-timeout
rescue in
413         # _handle_poll_timeout). If it is genuinely absent, the process is gone and no
marker will
414         # EVER appear ? fail fast + diagnostically rather than wait out the whole poll
budget.
415         marker = self._read_marker(done_uri)
416         if marker is not None:
417             return marker
418         raise RemoteExecutionFailed(
419             f"cloud-run: job={self._job_name} execution={execution or '<unknown>'}
reached "
420             f"terminal state {state!r} with NO completion marker ? the worker exited
before "
421             "writing one (crash / OOM / pre-marker abort, or a failed marker upload on
an "
422             "otherwise-Succeeded run). The execution is over, so the marker will never
appear; "
423             "inspect the Cloud Run Job execution log to diagnose: "
424             f"`gcloud run jobs executions describe {execution or '<execution>'} "
425             f"--region {self._region}`."
426         )
427
428     def _terminal_execution_state(self, execution: str) -> Optional[str]:
429         """Best-effort wrapper over the client's ``execution_terminal_state`` probe. A
raising or
430         flaky status API must NEVER kill a healthy wait (the done-marker stays the sole
authority),
431         so any exception ? and an empty/unresolvable execution name ? degrades to
``None`` (unknown
432         ? keep polling) and is DEBUG-logged, exactly today's marker-only behaviour."""
433         if not execution:
434             return None
435         try:
436             return self._client.execution_terminal_state(execution)
437         except Exception: # noqa: BLE001 ? advisory probe: never let it kill the
bucket-poll
438             LOG.debug(
439                 "cloud-run: execution status probe failed for %s ? treating as unknown",
440                 execution,
441                 exc_info=True,
442             )
443             return None
444
445     def _handle_poll_timeout(
446         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
447     ) -> dict:
448         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
449
450         The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
451         ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
452         ONE final marker check rescues that completed GPU run instead of wasting it. If

```

```

the marker
453         is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
454         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
455         execution name so the operator can verify the state in the GCP console. A cancel
failure
456         NEVER masks the original timeout."""
457         try:
458             late = self._read_marker(done_uri)
459         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
460             LOG.warning(
461                 "cloud-run: final post-timeout marker check failed ? treating as
absent",
462                 exc_info=True,
463             )
464             late = None
465         if late is not None:
466             LOG.warning(
467                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
468                 "? rescuing the completed result",
469                 self._job_name,
470                 execution,
471             )
472             return late
473         try:
474             self._client.cancel_execution(execution)
475         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
476             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
477             LOG.warning(
478                 "cloud-run: best-effort cancel of execution=%s failed: %s",
479                 execution,
480                 cancel_exc,
481                 exc_info=True,
482             )
483         else:
484             cancel_note = "cancel requested"
485             LOG.info("cloud-run: requested cancel of execution=%s", execution)
486             raise PolicyTimeout(
487                 f"{timeout_exc} [cloud-run job={self._job_name} "
488                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
489                 "state in the GCP console]"
490             ) from timeout_exc
491
492         # --- storage-backed completion signal (bucket-poll) ---
493         def _read_marker(self, done_uri: str) -> Optional[dict]:
494             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
495             ref = parse_uri(done_uri)
496             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
497                 return None
498             return (
499                 self._read_json(done_uri) or {}
500             ) # empty dict still terminates the poll (present == done)
501
502         def _read_json(self, uri: str) -> dict:
503             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
504             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
505             # would silently return {} for every gs://|s3:// object (the only backends this

```

```

shim targets).
506         try:
507             ref = parse_uri(uri)
508             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
509                 dst = os.path.join(td, ref.name or "obj.json")
510                 local = fetch(uri, dst, config=self._storage_config)
511                 with open(local, "r", encoding="utf-8") as f:
512                     data: Any = json.load(f)
513                 return data if isinstance(data, dict) else {}
514         except Exception:
515             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
516         return {}
517

```

13. assistant (2026-07-08T06:19:52.746Z)

I now have the complete picture. Let me trace the claim's causal chain against the actual code.

The claim's scenario: ``compute_target=cloud_run` + `phase_placement.compile=cloud_run_l4` + `storage.output_root=/data/reels`` (non-URI local path). The claimed outcome is that the worker stitches, ``reel_object_uri`` returns `""` so the mirror is skipped, the reel is discarded, yet the worker writes a ``status=success/rc=0`` marker ? silent success ? 404 download.

The decisive flaw in the claim is that a non-URI ``output_root`` doesn't just break the reel mirror ? it breaks the spec staging that the worker depends on, and the two are inseparable:

1. ``_dispatch_compile_remote`` builds ``spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"`` = ``/data/reels/_compile/<token>/spec.json`` (orchestration.py:1384), then ``storage.put(local_spec, spec_uri, ...)`` (orchestration.py:1391). For a non-URI path, ``parse_uri`` routes to the `**local**` backend ? the spec is written to the `**control-plane's local disk**`.
2. ``compute_target=cloud_run`` means ``get_compute_backend`` returns a real ``CloudRunCompute`` using ``GoogleCloudRunJobClient`` (compute/___init___py:90, cloud_run.py:109), which dispatches a `**separate, isolated Cloud Run Job container**` with ``--spec-uri /data/reels/_compile/<token>/spec.json``.
3. On that remote worker, ``cmd_compile`` calls ``spec = _read_compile_spec(args.spec_uri, ...)`` at worker/___main___py:490 ? `**not**` wrapped in try/except. ``_read_compile_spec`` does ``storage.fetch(spec_uri, ...)`` (worker/___main___py:426) on a CP-local path that does not exist on the worker ? it raises ? ``cmd_compile`` propagates the exception ? the worker exits `**without ever reaching `_stitch_reel` (line 506) or `_write_compile_marker` (line 523)**`. No success marker is written.
4. The dispatcher's ``run_compile`` bucket-polls; ``_poll_check`` finds no marker, the Executions API reports the execution terminal, and it raises ``RemoteExecutionFailed`` (cloud_run.py:390-426). That propagates out of ``run_compile`` and is caught in ``_dispatch_compile_remote`'s `except Exception`` (orchestration.py:1428-1434), returning ``{status: "failed", ...}``.

So the realistic outcome is a `**clean failed job**`, not a silent success. The claim's load-bearing premise ? "the worker still writes a marker `status=success/rc=0`` ? is false: the very condition that makes ``reel_object_uri`` return `""`` (a non-URI root) also stages the spec to a CP-local path the remote worker cannot read, so the worker crashes before it can stitch anything. The reel is never even produced, let alone silently discarded.

The only way the claim's chain would hold is if the control-plane and the Cloud Run worker shared the identical local filesystem at ``/data/reels`` (same box, or an exotic shared mount) ? which contradicts what ``compute_target=cloud_run`` means (a real, isolated remote container via ``GoogleCloudRunJobClient``). The reviewer did not establish that, and it is not how a `cloud_run` deployment is configured. Note also that this truthiness-only ``out_root`` check mirrors the pre-existing detect path (``_maybe_dispatch_remote``, orchestration.py:615-621), which fails the same clean way ? so it's not a new invariant break introduced by #521.

Per the instruction to default to `is_real=false` when the failure can't be concretely confirmed,

and having concretely traced the realistic path to a clean failure, I'm refuting this claim.

14. user (2026-07-08T06:20:03.492Z)

Structured output provided successfully